



Robust vulnerability detection with limited data via training-efficient adversarial reprogramming

Zhenzhou Tian^{1,2,3} · Chuang Zhang¹ · Yunpeng Hui¹ · Jiaze Sun^{1,2,3} · Yanping Chen^{1,2,3} · Lingwei Chen⁴

Received: 26 February 2025 / Accepted: 4 January 2026

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2026

Abstract

The substantial increase in software vulnerabilities poses a significant threat to system security, prompting a surge of interest in applying deep learning (DL) to vulnerability detection. However, current DL-based detectors heavily rely on large-scale labeled data, leading to inefficiency and notable performance degradation in scenarios with limited data. Furthermore, these detectors often lack robustness against adversarial code transformation attacks. To address these challenges, this paper proposes ArVD (Adversarial Reprogramming-Based Vulnerability Detector), which implements a novel and computationally inexpensive approach to reprogram a pre-trained model for detecting vulnerabilities at the function level. Specifically, ArVD first constructs structure-aware token sequences from source code. Given these inputs, the model then exclusively learns universal perturbation elements to be added into the token sequences and leverages self-attention mechanism to enhance non-linear interactions among tokens and perturbations, such that the learning capabilities from the pre-trained model can be adapted to vulnerability detection with less training data and time yet higher detection effectiveness and robustness. Extensive experiments conducted on multiple datasets demonstrate that ArVD significantly reduces the trainable parameters to approximately 20,000 while outperforming DL-based baselines in terms of detection effectiveness, data-limited performance, as well as runtime overhead. Moreover, ArVD effectively counters code transformation attacks; compared to the state-of-the-art ZigZag framework that is designed to enhance detector robustness, ArVD exhibits an averaged relative improvement of 18.89% in F1, with a decrease of 43.25% and 42.65% in FPR and FNR, respectively.

Keywords Vulnerability detection · Adversarial reprogramming · Model robustness · Code transformation · Data-limited training

Extended author information available on the last page of the article

Published online: 19 January 2026

Springer

1 Introduction

There has been a noticeable surge in software security attacks in recent years, while this surge is largely driven by a substantial increase in software vulnerabilities. To put it into perspective, the CVE website (CVE 2025) recorded approximately 5,200 vulnerabilities in 2013—a number that has dramatically escalated to more than 240,000 by 2025. Given the significant threats posed by these vulnerabilities, both academia and industry have been dedicated to developing effective models for detecting them. Especially, with the revolutionary learning structures and capabilities offered by deep learning (DL), various data-driven methods (Chakraborty et al. 2021; Wen et al. 2024a; Steenhoek et al. 2023; Cheng et al. 2024; Wu et al. 2024) have emerged to enable efficient and precise vulnerability identification in source code.

In the line of DL-based vulnerability detection, classifiers are constructed using labeled samples to extract semantic-aware features and automatically learn vulnerability-specific characteristics, thereby facilitating the identification of hidden vulnerabilities within projects. For example, VulDeePecker (Li et al. 2018) employs a BiLSTM to capture the vulnerable features from the code gadgets sliced based on potential vulnerability key points, such as library function calls (Li et al. 2018). TrVD distills the features implied in each subtree segmented from the AST with a tree-structured neural network, and then utilizes a transformer-style encoder to aggregate them across all subtrees to indicate vulnerability (Tian et al. 2024). Devign (Zhou et al. 2019), a graph-based detector, mixes AST, CFG, Data Flow Graph (DFG), and code sequences into a joint graph as inputs, and employs a Gated Graph Neural Network (GGNN) to aggregate node information, thus boosting vulnerability detection performance.

Despite receiving widespread attention, DL-based vulnerability detectors still face three major limitations. (1) These detectors heavily rely on large-scale labeled data, which may lead to performance degradation, particularly in terms of poor generalization to unseen data, under limited data conditions (Yang et al. 2023; Croft and Babar 2023). This poses a significant challenge for effectively applying these models. (2) Existing DL models require considerable training effort in terms of time consumption. For instance, the SySeVR model takes an average of 7.898 seconds for data processing and model training on an individual sample (Wu et al. 2022). (3) DL detectors are faced with the learning-security challenge of lacking adversarial robustness and remain susceptible to intentionally designed perturbations on input samples, thus suffering from drastic drop in performance in terms of high false positive rate (FPR) and false negative rate (FNR) as well as low F1 scores (Li et al. 2021a).

Due to the inherent vulnerability to adversarial attacks, adversarial reprogramming (Elsayed et al. 2018) has been proposed to repurpose an ML model trained in a source domain to perform a target-domain task through learning a universal perturbation to the target-domain data without modifying the pre-trained model parameters, where the domains and tasks can be completely different (Chen et al. 2022). Unlike traditional adversarial attacks that exploit the linear approximations of ML models to mislead classifications, adversarial reprogramming relies entirely on the nonlinear interactions of the input and the perturbation. In other words, given a DL model of non-linear structure, a well-formulated offset added to its input would be sufficient to

repurpose the model for a new task without requiring model fine-tuning. This yields some significant advantages: (1) only the perturbation is trainable that may take less model training effort in terms of labeled data and training time yet better leverage of the pre-trained model's learning capability than transfer learning and training from scratch; (2) reprogramming by adding adversarial perturbation to the input serves as an equivalent to adversarial training that may enhance the model robustness against potential adversarial data manipulations.

Inspired by these advantageous properties and successful applications of adversarial reprogramming in image and text domains (Tsai et al. 2020; Neekhara et al. 2019; Chen et al. 2022), we propose a novel and computationally inexpensive model that implements adversarial reprogramming for vulnerability detection (ArVD) at the function level, so as to mitigate the aforementioned limitations. ArVD appends an input transformation layer and a label remapping layer to a pre-trained language model (PLM) to form the reprogrammed model for vulnerability detection. More specifically, this model first constructs the AST representation of the target code snippet, and then linearizes the source code into a token sequence in a structure-aware manner during tree traversal, enabling the model to better understand and analyze the target source code. Unlike conventional vulnerability detection models that entail training or fine-tuning of all model parameters, ArVD exclusively learns universal perturbations to be added into the input token sequence and leverages self-attention mechanism to enhance model nonlinearity and modulate the way the input is processed; this allows for seamless transfer of learning capabilities from the pre-trained model to vulnerability detection with less training data and time yet higher detection effectiveness and robustness. Moreover, in contrast to existing adversarial reprogramming techniques that often incorporate full-scale perturbations that still represent a large number of parameters to update (Neekhara et al. 2022; Elsayed et al. 2018; Tsai et al. 2020; Chen et al. 2022), ArVD inserts token-level perturbations to drastically decrease the trainable parameters, making it more suitable for vulnerability detection with limited data. To derive the final output, ArVD further employs label remapping that elaborates a linear transformation to convert the prediction scores from the original task into vulnerability detection results. In this process, the perturbation elements can be optimized by minimizing the loss function using Adam. This paper has the following major contributions:

- We propose a novel adversarial reprogramming model, ArVD, aiming to achieve precise and efficient vulnerability detection through the exclusive optimization of perturbation elements.
- Three distinctive technical designs are introduced to implement ArVD: (1) a structure-aware approach is leveraged to linearize the source code into a token sequence, preserving both code semantics and structure; (2) token-level perturbations are incorporated into the input sequences, substantially reducing the number of parameters to update; and (3) label remapping is elaborated to convert the output of the original task to the target task, effectively deriving the vulnerability detection results.
- Extensive experiments are conducted on multiple datasets to assess the performance of ArVD in vulnerability detection. Compared to other state-of-the-

art models, the results demonstrates that (1) ArVD significantly improves the detection performance, especially on small datasets with an average absolute increase of 6.70% in accuracy and 8.28% in F1 score; (2) ArVD enables perturbation adjustment that decreases the trainable parameters to approximately 20,000, which contributes to reduced time cost, making ArVD 3.2 times and 2 times faster than full fine-tuning and LoRA fine-tuning of the UniXcoder model; and (3) ArVD also exhibits enhanced robustness against adversarial code transformation attacks with a relative improvement of 18.89% in average F1 score, a decrease of 43.25% in FPR, and a decrease of 42.65% in FNR, which is superior to those robust models trained using the ZigZag framework. To facilitate relevant researches, the source code of ArVD have been made publicly available at <https://github.com/XUPT-SSS/ArVD>.

The rest of the paper is organized as follows: Section 2 explores the preliminaries and motivation of leveraging adversarial reprogramming. Section 3 explains in detail the core design of ArVD. Section 4 presents the experimental evaluation and analysis. Section 5 discusses potential threats, limitations, and some interesting future extensions. Related work is reviewed in Section 6. Finally, Section 7 concludes.

2 Preliminaries and motivation

In this section, we first provide insight into adversarial reprogramming technique, and then delve into the motivation behind our proposed ArVD model.

2.1 Adversarial reprogramming

Adversarial reprogramming is designed to achieve a more challenging goal than adversarial attack, specifically “reprogramming the model to perform a task chosen by the attacker, without the attacker needing to compute the specific desired output” (Elsayed et al. 2018). Although both adversarial attacks and adversarial reprogramming involve adding perturbations, their purposes differ. Adversarial attacks aim to degrade a model’s performance or mislead it into producing an incorrect output (e.g., causing a vulnerability detector to classify a vulnerable function as non-vulnerable) by manipulating the input with adversarial perturbations. In contrast, adversarial reprogramming seeks to repurpose the model to perform a new task rather than generate a specific hardcoded output. This is typically achieved by appending an input transformation layer that adds learnable universal perturbations to the input and an output remapping layer that converts the prediction from the original task into the target task at the front and back of the original model, respectively, forming the complete reprogrammed model.

Formally, given a model $s(\cdot)$ trained to perform an original task, for the input sample $x_s \in \mathcal{X}_s$, it outputs the class label $y_s \in \mathcal{Y}_s$ denoted as $s : x_s \rightarrow y_s$. Adversarial reprogramming executes the adversarial task of model $t(\cdot)$, and for the task sample $x_t \in \mathcal{X}_t$, it outputs the class label $y_t \in \mathcal{Y}_t$ denoted as $t : x_t \rightarrow y_t$, which is accomplished by repurposing $s(\cdot)$ through input transformation $f_s(\cdot, \theta)$ and label remapping $f_t(\cdot)$ that

associate the two tasks. The input transformation $f_s : (x_t, \theta) \rightarrow x_s$ transforms the input space of the adversarial task $\mathcal{X}_t$ to the input of the original task $\mathcal{X}_s$ by learning a universal perturbation θ , such that the label remapping $f_t : s(f_s(x_t, \theta)) \rightarrow y_t$ can map the class space of the original task $\mathcal{Y}_s$ back to the labels of the target task $\mathcal{Y}_t$. Accordingly, these operations can be summarized as follows:

$$x_s = f_s(x_t, \theta), x_t \in \mathcal{X}_t, x_s \in \mathcal{X}_s \quad (1)$$

$$y_s = s(x_s), y_s \in \mathcal{Y}_s \quad (2)$$

$$y_t = f_t(y_s), y_t \in \mathcal{Y}_t \quad (3)$$

θ represents the only trainable parameter. The ultimate goal of adversarial reprogramming is to update θ by solving the following optimization problem:

$$\hat{\theta} = \underset{\theta}{\operatorname{argmin}} (-\log p(f_t(y_s) | f_s(x_t, \theta))) \quad (4)$$

where $p(f_t(y_s) | f_s(x_t, \theta))$ indicates the probability that an input x_s transformed from x_t being classified as y_s , which can be mapped to y_t .

2.2 Motivation behind ArVD

While DL models have been successfully deployed to detect vulnerabilities, constructing and maintaining these models represents significant challenges in terms of data scarcity, computational cost, and model robustness.

Data scarcity Obtaining labeled vulnerable code in real world is challenging (Nong et al. 2023b, a; Tian et al. 2025; Dolan-Gavitt et al. 2016), not to mention the quantity of samples attributed to a particular vulnerability type. Figure 1 depicts the statistical results on the vulnerable samples distributed across 85 Common Weakness Enumeration (CWE) categories (CWE 2025) as summarized in TrVD (Tian et al. 2024). Notably, the sample counts for half of these categories are less than 100, with 27 categories having fewer than 50 samples. This contradicts with the majority of current DL-based

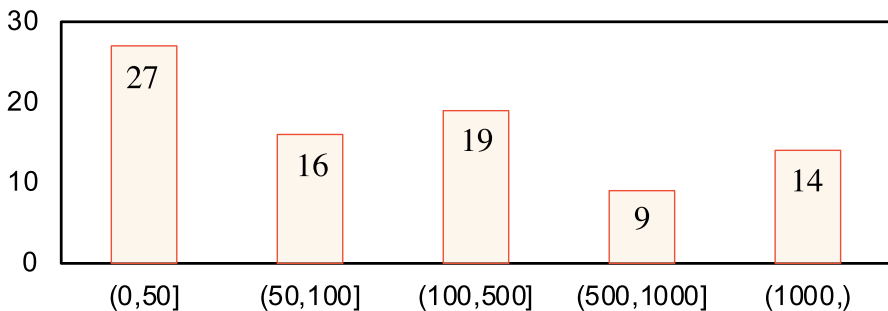


Fig. 1 Distribution of CWE categories based on sample size

vulnerability detectors, which heavily rely on large-scale labeled data. For example, Reveal (Chakraborty et al. 2021) achieved an accuracy of 86.29% on the TrVD dataset, but the average accuracy dropped to 72.61% in data-limited scenarios.

Computational cost Conventional mechanisms for constructing DL-based vulnerability detection models often require training models from scratch or fine-tuning pre-trained models. This leads to significant computational overhead. Taking the SySeVR model (Li et al. 2021c) as an example, the average computational time per sample extends up to 7.898 seconds, which heavily limits its data processing capability. This not only impacts the effectiveness of vulnerability detection but also contradicts the current advocacy of eco-friendly practices, as these models consume a considerable amount of computing resources.

Model robustness ML is intrinsically susceptible to adversarial attacks, while DL-based vulnerability detectors are no exception. Recent researches (Li et al. 2021a; Yu et al. 2023) have indicated that these DL-based detection models, though varying in learning structures and input representations, may still fail when facing adversarial code transformations. For instance, the detector using BiGRU trained on token sequences within program slices suffers from a FPR increase from 7.0% to 21.8% and a FNR skyrocketing from 9.9% to 74.8% under attack (Li et al. 2021a). This underscores the importance of considering the robustness aspect when constructing DL-based vulnerability detectors (Rahman et al. 2024).

Adversarial reprogramming for vulnerability detection To address aforementioned challenges, ArVD leverages adversarial reprogramming to repurpose a pre-trained language model for vulnerability detection over structure-preserving token sequences extracted from code snippets. Adversarial reprogramming requires the nonlinear interactions of the input x and the perturbation θ , which can be fulfilled by the language model of attention structure Vaswani et al. (2017). More specifically, consider such a language model receiving an input $x_s = f_s(x_t, \theta) = x_t + \theta$ transformed from a task sample x_t with the perturbation θ . The output of an attention operation can be approximated as:

$$\text{Attention}(x_t + \theta) = \text{output}_{x_t} + \text{output}_{\theta} + \text{output}_{\theta^T x_t} \quad (5)$$

where the pre-trained language model can be adapted through $\text{output}_{\theta^T x_t}$ without modifying the original model parameters. Thus, adversarial reprogramming retains all original parameters of the pre-trained model unchanged, while solely learning the perturbation θ to process input and perform the adversarial task. This implies that adversarial reprogramming can benefit our vulnerability detection problem in three key aspects:

- As the downstream task differs completely from the original one, transfer learning demands a substantial amount of vulnerability samples to fine-tune the extensive pre-trained model parameters. In contrast, adversarial reprogramming exclusively updates the perturbation θ , thus notably reducing the model complexity

and the need for labeled data and computational cost. For instance, ArVD with UniXcoder backbone, has only approximately 20,000 learnable parameters during model training, which is about $6,250\times$ smaller in learnable parameter size and $3.2\times$ faster in training time compared with typical full fine-tuning.

- Vulnerability-specific information tends to be subtly embedded within code, while the pre-trained language model provides powerful learning capabilities to extract intricate semantic patterns from token sequences, potentially achieving high performance in vulnerability detection.
- During adversarial reprogramming, input token sequences undergo offsetting by adversarial perturbation, which regularizes model optimization in a way similar to adversarial training, thereby amplifying model robustness against potential code transformations.

3 Design of ArVD

In this section, we present the technical design of ArVD in detail, the overview of which is illustrated in Fig. 2. ArVD proceeds with the following main steps: (1) input preparation, which converts the given code snippets into structure-aware token sequences as inputs; (2) input transformation and non-linear interaction, which introduce trainable perturbation elements into the input token sequences, so as to participate into the self-attention computation enforced in the PLM to adapt its way of handling the original code token sequences; (3) label remapping, which maps the output of the original task to the vulnerability detection results; and (4) optimization, which updates the perturbation elements and the label remapping layer to adapt the language model for vulnerability detection. After training, the input transformation layer, the original PLM, and the label remapping layer together form the complete reprogrammed model, which accepts code functions and predicts their vulnerabilities.

3.1 Input preparation

3.1.1 Code normalization

To promote the learning of code semantics that are indicative to vulnerabilities, it is necessary to normalize the source code and eliminate noisy information. Similar to existing works (Tian et al. 2024; Wu et al. 2022), we begin by removing comments from the code, followed by replacing each user-defined variable and function name

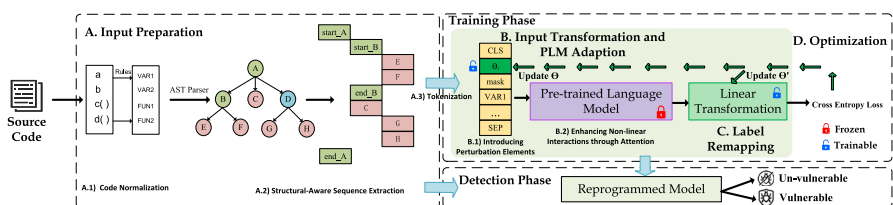


Fig. 2 The overview of ArVD

with *VAR_i* and *FUN_i* respectively, where *i* denotes their first appearance order in the code snippet. The purpose of this normalization process is to mitigate the risk of the model relying on function names for identification rather than detecting underlying vulnerability patterns. For instance, in the SARD dataset, function names are often embedded with vulnerability-related information, such as “good” and “bad”. Consequently, the model might directly leverage these names to predict the presence of vulnerabilities without exploring other code semantics. By employing normalization, we ensure that the model emphasizes the detection of actual vulnerability patterns, thereby enhancing its robustness and generalization capability. Figure 3 illustrates an example of the normalization process applied to a code snippet. Throughout this process, other essential tokens such as keywords, library function names, and punctuation remain unchanged. This helps preserve information pertinent to code semantics and prevents the loss of crucial semantic content during normalization, ensuring the reliability of the entire model.

3.1.2 Structure-aware sequence extraction

To facilitate reprogramming a pre-trained language model and minimize detection overhead, ArVD opts to operate on the sequence of code tokens. However, simply lexically parsing the source code into a token sequence would inevitably discard its structural and logical information (Tian et al. 2024), making it harder to reveal and accurately understand the code semantics in subsequent stages. Considering that structural information can be derived from the AST of the source code, we initially convert the source code to an AST where key code elements such as variables, symbols, and operators are represented as leaf nodes, and a series of governed code structures such as expressions and loops are represented as non-leaf nodes; subsequently, a structure-aware approach is employed to linearize the AST into a code structure-preserving token sequence.

Specifically, we adopt tree-sitter (Tree-sitter 2025) to construct the AST from a given function, which uses *function_definition* to designate the root of the function. Commencing from the root, we apply the X-SBT (Niu et al. 2022) to linearize the corresponding tree into a token sequence that retains the code structure. This

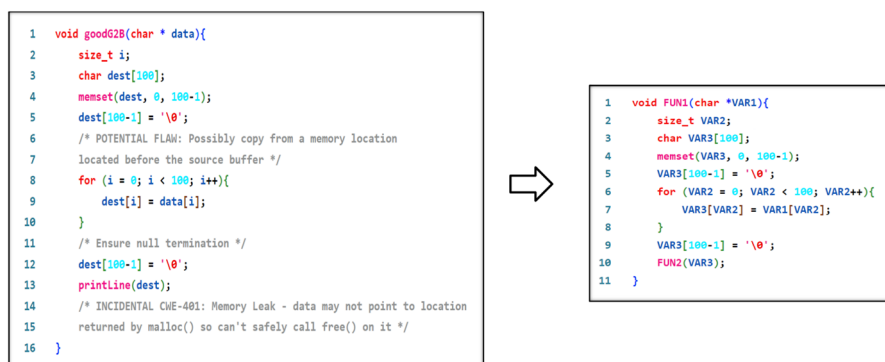


Fig. 3 An normalization processing example that is performed on a code snippet

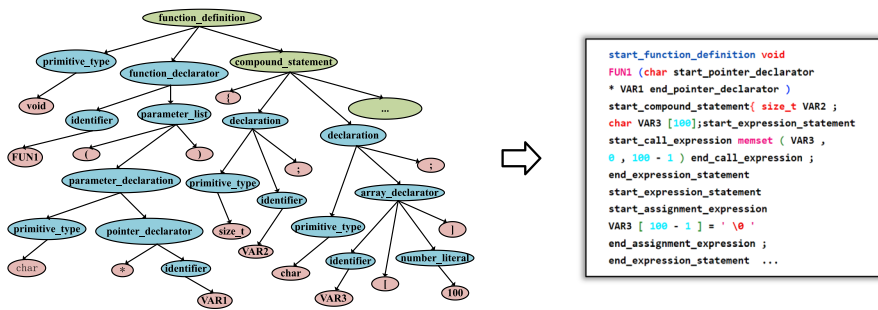


Fig. 4 An example of extracting structure-aware sequences on code snippets

approach improves upon SBT by reducing the sequence length by over half. On top of that, we further improve the X-SBT by prefixing non-leaf nodes with “start_” and “end_” markers to indicate their structural parent-child and sibling relationships in the serialized token sequence representation. Compared to the obscure XML-style format used in the original X-SBT (Niu et al. 2022), explicitly marking the start and end of code structures with these clearer and more meaningful indicators helps the model become more aware of the relationships and contextual dependencies between code segments. This is particularly beneficial when reprogramming a pre-trained language model, as these more detailed tokens with indicative meanings are expected to align better with the input requirements of the model’s inner transformer-style encoders, thereby improving its overall semantic parsing capability. Figure 4 illustrates this process with a simple example.

3.1.3 Tokenization

To avoid vocabulary inflation posed by different identifiers and improve learning performance, subword tokenization algorithms are employed to decompose words into smaller subwords or characters. These algorithms are commonly applied in managing complex vocabularies, particularly in the fields of NLP and code analysis, which help alleviate out-of-vocabulary (OOV) challenges and improve model coverage and vocabulary generalization. RoBERTa (Liu et al. 2019), a pre-trained model, utilizes BBPE tokenizer, specifically designed for handling binary data or text in specialized domains like code. Trained on a large-scale dataset, RoBERTa boasts a vast vocabulary of 50,000 subwords, which is sufficient to handle the requirements of code normalization and mitigate OOV issues. Due to such impressive properties and advantages, our model uses the RoBERTa vocabulary for tokenization processing, generating the final input token sequences.

3.2 Input transformation and PLM adaption

Adversarial reprogramming designed in ArVD repurposes the target pre-trained language model to execute vulnerability detection task through an input transformation function and a label remapping function, which is illustrated in Fig. 5. Input transformation is to convert the token sequences into the input space of the

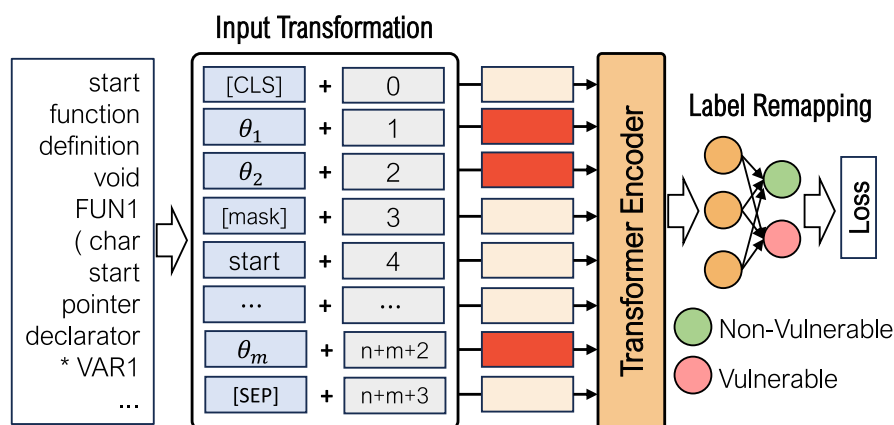


Fig. 5 Adversarial reprogramming via input transformation and label remapping

language model by introducing perturbation elements to be added the sequences, which are then fed to the model for layer-wise non-linear interactions via self-attention mechanism. This adapts the model to generate the outputs that can be further processed by the label remapping.

3.2.1 Introducing perturbation elements

Existing adversarial reprogramming techniques tend to add full-scale perturbations to image inputs to reshape the feature extraction capabilities of pre-trained models, including ImageNet networks (Elsayed et al. 2018; Tsai et al. 2020; Chen et al. 2022), and Vision Transformer (ViT) (Neekhara et al. 2022; Li et al. 2023). Despite promising results, these models face limitations as the perturbation represents approximately 150K-338K parameters to update; consequently, the models still demand extensive training efforts in terms of labeled data and computational cost, which is unsatisfactory for our vulnerability detection task. Recently, adversarial reprogramming has been generalized to text classification tasks (Neekhara et al. 2019). However, this method operates in the text space and resembles an end-to-end adversarial attack, making it computationally expensive to derive word perturbations.

Drawing inspiration from task-specific prompt designs (Li and Liang 2021; Lester et al. 2021), in this paper, we propose to formulate token-level perturbations in the embedding space that are inserted into the token sequences and perform gradient-based optimization to update them. Formally, these perturbation elements are defined as $\Theta = \{\theta_1, \theta_2, \theta_3, \dots, \theta_m\}$, where m is a hyperparameter determining their quantity. Considering a token sequence $X_t = \{x_{t_1}, x_{t_2}, \dots, x_{t_n}\}$ that has already been tokenized with each token being embedded to a 768-dimensional vector, where n is the sequence length, the input sequence X_s for the pre-trained

language model can be constructed by inserting perturbation elements and an additional token $[mask]$ into the token sequence X_t , which is represented as

$$X_s = f_s(X_t, \Theta) = \{[cls], \theta_1, [mask], x_{t_1}, \dots, x_{t_n}, \theta_m, [sep]\} \quad (6)$$

where $|X_s| = n + m + 3$ and $f_s(X_t, \Theta)$ represents a specific perturbation template, which specifies both the number of perturbation elements to insert and the specific positions to insert them. Let L denote the maximum input sequence length for the pre-trained model. Sequences longer than L will be truncated, while sequences shorter than L are filled with special padding tokens (e.g., $[PAD]$), which the model will ignore and not compute. These perturbation elements can be independently placed before, within, or after the sequences, depending on the specified template. In our implementation, once the number of perturbation elements is determined, we randomly pick the corresponding number of insertion positions to form a specific perturbation template. This template is then applied to both the training samples during the model training phase and the test samples during the model inference phase. This design not only offers more degrees of freedom, but also significantly reduces the number of trainable parameters to $m \times 768$ (we expect m to be a small value). The impact of the number of the perturbation elements m on the model performance will be evaluated in Section 4.2.4. It is worth noting that the inclusion of $[mask]$ in the input sequence is intended to use the feature vector at the $[mask]$ position as the representation of the entire sequence, serving as the output of the pre-trained model.

As the input transformation is conducted in the embedding space, we can readily utilize gradient-based optimization to find the optimal perturbation elements. Furthermore, only the perturbation elements Θ within the input sequence X_s are trainable, while all the embeddings and positions for the original tokens remain unchanged. In this respect, the formulation of X_s thus yields the additional advantage of expediting the convergence of perturbation updates using batch processing, thereby enhancing the flexibility of our model.

3.2.2 Enhancing non-linear interactions through attention

Our goal is to leverage the perturbation elements inserted into the token sequences to change the way these sequences are processed by the pre-trained model, and fine-tune these perturbations to adapt the model for vulnerability detection task. This necessitates the non-linear interactions between the perturbation elements and the original tokens, which can be regulated by the multi-head self-attention mechanism implemented in the pre-trained language model. Specifically, the attention layer dynamically learns and allocates attention weights to determine the relevance between perturbation elements and different tokens. By taking advantage of this attention mechanism, ArVD is able to (1) selectively emphasize or weaken different parts of the input sequence, thereby better capturing vulnerability-specific

semantics and detecting vulnerabilities; and (2) identify the most effective perturbations for model adaptation, hence improving the accuracy and robustness of the model. The core operations in the multi-head attention layer involve the self-attention score calculations, which can be computed within each head with the following equations:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (7)$$

$$Q = W^Q X_s, K = W^K X_s, V = W^V X_s \quad (8)$$

$\text{Attention}(Q, K, V) \in \mathbb{R}^{(m+n+3) \times d_v}$ is the attention score matrix, where each row g_i represents the d_v -dimensional attention vector for the corresponding row in the input matrix X_s ; $Q \in \mathbb{R}^{(m+n+3) \times d_k}$, $K \in \mathbb{R}^{(m+n+3) \times d_k}$ and $V \in \mathbb{R}^{(m+n+3) \times d_v}$ are three matrices obtained by multiplying the input X_s with weight matrices that have been pre-trained in the selected language model. $\sqrt{d_k}$ serves as a scaling factor to stabilize gradients by preventing excessively large softmax results. Following multiple attention layers, the feature representation at the *[mask]* position is extracted as the output of the pre-trained model for the input sequence $X_s \in \mathcal{X}_s$, represented as $s(X_s) \in \mathbb{R}^d$, which are subsequently fed for label remapping.

3.3 Label remapping

Label remapping is to map the output of the pre-trained language model to the class space of the target task in order to perform our selected vulnerability detection task. This process is more straightforward than input transformation. Previous works on adversarial reprogramming often utilize either hard-coded or soft-coded mapping to directly associate two different class spaces in a non-parametric manner (Elsayed et al. 2018; Neekhara et al. 2022; Tsai et al. 2020; Chen et al. 2021, 2022). However, these approaches are infeasible in our application scenario, due to the fact that the output $s(X_s)$ derived from the pre-trained model (as discussed in Section 3.2.2) is a d -dimensional feature representation rather than class logit vector. In other words, to obtain the final class output, the label remapping needs to be implemented using a linear transformation, which can be formulated as:

$$f_t(s(X_s)) = \Theta'^T s(X_s), \Theta' \in \mathbb{R}^{d \times 2} \quad (9)$$

where Θ' is a learnable weight matrix. The additional parameters introduced by this linear layer are minimal and do not significantly increase training efforts and computational cost. The vulnerability detection probabilities can be calculated as:

$$p(y_t | X_t) = \frac{\exp(f_t(s(X_s))_{y_t})}{\sum_{i \in \mathcal{Y}_t} \exp(f_t(s(X_s))_i)} \quad (10)$$

where $\mathcal{Y}_t$ is the set of classes (i.e., $\{0:\text{non-vulnerable}, 1:\text{vulnerable}\}$). Θ' is optimized with perturbation elements Θ .

Algorithm 1 Training algorithm for ArVD.

Input: Training code snippets ($\mathcal{X}_t, \mathcal{Y}_t$), perturbation template: $f_s(\cdot, \Theta)$, pre-trained model $s(\cdot)$.
Output: Model parameters $\hat{\Theta}$.

for each code snippet $x_t \in \mathcal{X}_t$ **do**
 Normalize x_t and construct AST;
 Extract structure-aware sequence from AST;
 Perform tokenization using RoBERTa vocabulary;
 Generate token sequence: $x_t = \{x_{t1}, x_{t2}, \dots, x_{tn}\}$;
 Get token embeddings: $\mathbf{X}_t = \{\mathbf{x}_{t1}, \mathbf{x}_{t2}, \dots, \mathbf{x}_{tn}\}$;
end
Initiate the perturbation elements Θ ;
for each training epoch $t \leq T$ **do**
 $\mathbf{X}_s = f_s(\mathbf{X}_t, \Theta) = \{[cls], \theta_1, [mask], \mathbf{x}_{t1}, \dots, \mathbf{x}_{tn}, \theta_m, [sep]\}$;
 Feed $\mathbf{X}_s$ to $s(\cdot)$ with attention (7, 8);
 Get model output $s(\mathbf{X}_s)$ at $[mask]$ position;
 Get class output $f_i(s(\mathbf{X}_s))$ (9, 10);
 Update $\hat{\Theta}$ by solving (12) using Adam;
end

3.4 Optimization

By combining input transformation and label remapping, the trainable parameter sets for ArVD can be finalized as:

$$\hat{\Theta} = \{\theta_1, \theta_2, \theta_3, \dots, \theta_m; \Theta'\} \quad (11)$$

The optimization of ArVD can be accordingly formulated as a cross-entropy loss between the true labels and prediction scores, incorporated with L2 regularization to improve generalization on the test samples, which is denoted as:

$$\Theta^* = \underset{\Theta}{\operatorname{argmin}} \left(- \sum_{\mathbf{X}_t \in \mathcal{X}_t} y_t \log p(y_t | \mathbf{X}_t) + \lambda \|\hat{\Theta}\|_F^2 \right), \quad (12)$$

where λ is the regularization parameter. As we have full access to the pre-trained language model, the optimal perturbation elements can be found by minimizing (12) using the gradient descent algorithm with Adam optimizer. Algorithm 1 outlines the training algorithm for ArVD.

4 Experimental results and analysis

In this section, we conduct extensive experiments to fully evaluate the performance of ArVD on the benchmark datasets and answer the following research questions:

- **RQ1: Performance on vulnerability detection.** How does the performance of

ArVD compare to state-of-the-art DL-based vulnerability detection models?

- **RQ2: Efficiency analysis.** How does ArVD’s runtime overhead compare to other detection models?
- **RQ3: Data-limited training.** How does ArVD perform with small training samples, and how does it compare to other vulnerability detection models?
- **RQ4: Impact of perturbation element number and pre-trained model.** To what extent do the number of perturbation elements and the underlying pre-trained model impact ArVD’s detection effectiveness?
- **RQ5: Impact of perturbation position.** To what extent do different perturbation placement strategies affect the detection performance of ArVD?
- **RQ6: Robustness evaluation.** Does ArVD exhibit robustness against code transformation techniques?

4.1 Experimental setup

4.1.1 Datasets

We evaluate ArVD on four benchmark vulnerability datasets, as summarized in Table 1. TrVD, Reveal, and Devign datasets are used to address the first five questions, while ZigZag dataset is used to evaluate model robustness.

- **TrVD** (Tian et al. 2024) is a dataset constructed for evaluating TrVD model, which is composed of C/C++ functions based on the Software Assurance Reference Dataset (SARD), a widely used vulnerability database.
- **Reveal** (Chakraborty et al. 2021) is a real-world dataset curated from the Linux Debian kernel and the Chromium to address issues of extensive data duplication and unrealistic distribution of vulnerable categories in existing datasets.
- **Devign** (Zhou et al. 2019) is a dataset comprising function-level C/C++ source code from projects QEMU and FFmpeg, which has been manually annotated and verified by security researchers to represent real-world scenarios.
- **ZigZag** (Li et al. 2021a) originates from NVD and SARD. 8 adversarial code transformation attacks were applied to 6,803 programs through Tigress, resulting in 50,562 vulnerable examples and 80,043 non-vulnerable examples.

It can be observed that the ratios of vulnerable to non-vulnerable samples are generally imbalanced across these datasets, particularly in Reveal and BigVul. Such class imbalance is well known to hinder the effective training of DL-based vulnerability

Table 1 Statistics of the datasets

Dataset	#Vulnerable	#NonVulnerable	Vul_Ratio (%)	Dataset Type
TrVD	98,181	166,641	37.07	Synthetic
Reveal	2,240	20,494	9.85	Real-World
Devign	12,460	14,858	45.61	Real-World
BigVul	10,900	177,736	5.78	Real-World
ZigZag	50,562	80,043	38.71	Hybrid

detectors (Yang et al. 2023; Chakraborty et al. 2021). To mitigate this issue, and following prior work (Hin et al. 2022; Li et al. 2021b), we adopt an undersampling strategy during training, where the number of non-vulnerable samples is reduced to match the number of vulnerable samples, thereby producing a balanced training set. For validation and testing, however, we retain the original class distribution to more accurately reflect real-world deployment conditions.

4.1.2 Implementation details

We set the initial learning rate to 0.0006, and select the best-performing model, Microsoft/UniXcoder, which is a pre-trained model for programming languages with a maximum input sequence length of 1,024, along with 27 perturbation elements. The adoption of UniXcoder and the number of perturbation elements are discussed in detail in Section 4.2.4. Perturbation elements are initialized using the embedding of *[mask]*. We utilize Adam optimizer with a slanted triangular schedule for the learning rate, with 6% warm-up steps, and conduct training for 10-20 epochs with a batch size of 16. This strategy expedites the convergence speed of the model and fine-tunes its parameters. Within each epoch, we calculate the accuracy on the validation set. If the validation accuracy doesn't notably improve for 5 consecutive epochs, we terminate the training process early. We then select the model showing the best performance on the validation set as the final model and evaluate its performance on the test set. All experiments are conducted on a Linux server featuring one NVIDIA RTX3090 GPU.

4.1.3 Baselines

We compare ArVD with advanced DL-based vulnerability detection baselines, including two widely adopted models: TokenLSTM and TokenCNN, eight state-of-the-art models: VulCNN, Reveal, TrVD, LineVul, EPVD, CAPTURE, HardVD, LIVABLE, as well as the UniXcoder_{fft} and UniXcoder_{lora}, which take the structure-aware token sequences as inputs but are trained using the typical full fine-tuning (FFT) and LoRA fine-tuning strategies, respectively. Besides, DeepSeek-V3.1 also serves as one baseline with prompt engineering. These baselines are briefly introduced as follows:

- **TokenLSTM** (Schuster and Paliwal 1997) is a LSTM variant that captures both forward and backward semantics from source code sequences by implementing bi-directional LSTMs and multiple MLP layers for vulnerability detection.
- **TokenCNN** (Russell et al. 2018) is a CNN variant that processes natural language texts rather than images. In vulnerability detection task, the source code is embedded as a vector input. The model captures semantic information through multiple convolutional layers and pooling layers. The outputs are concatenated into a single vector, which is then fed into the softmax layer for classification.
- **VulCNN** (Wu et al. 2022) also operates on the PDG, but embeds its nodes with Sent2Vec, which treats PDG as a social network and applies centrality analysis to measure the importance of different elements in the graph by calculating degree centrality, Katz centrality, and closeness centrality. These measures are then mapped to three channels of an image and fed into CNN for vulnerability detection.

- **Reveal** (Chakraborty et al. 2021) operates on Code Property Graphs (CPG), which employs GGNN to extract vulnerability-specific features in the code, and adopts resampling techniques (Chawla et al. 2002) and triplet loss (Mao et al. 2019) to improve the vulnerability detection performance.
- **TrVD** (Tian et al. 2024) is a tree-based model, which extracts features from the AST with attention-augmented tree-structured recursive neural network for vulnerability detection.
- **LineVul** (Fu and Tantithamthavorn 2022) performs BPE subword tokenization to obtain a sub-token sequence for a function. Each sub-token and its position within the sequence are then embedded into a word encoding vector and a positional encoding vector, respectively. These vectors are then concatenated and fed into the pre-trained CodeBERT for feature extraction and vulnerability prediction.
- **EPVD** (Zhang et al. 2023) constructs a syntax-based CFG and decomposes it into multiple paths. CodeBERT and a CNN are then leveraged to extract features from them with intra- and inter-path attention. These vectors are then combined and fed into an MLP for vulnerability detection.
- **CAPTURE** (Tian et al. 2025) converts the code token sequence into a perturbation image by reshaping each token's embedding from a learnable universal perturbation dictionary, on which basis the ViT model is reprogrammed by appending a dynamic label remapping scheme that reassigns the model's output to the binary vulnerability detection result.
- **HardVD** (Tian et al. 2025) also adopts cross-modal adversarial reprogramming as CAPTURE for vulnerability prediction. However, it introduces a line encoder for capturing key features from per line of code, which are then organized as patches to create a semantic image. The convolutional filters in the line encoder meanwhile act as universal perturbations, enabling the ViT model to be adapted for vulnerability detection through a non-parametric label remapping scheme.
- **LIVABLE** (Wen et al. 2024b) employs a differential propagation-based GNN to extract features from the code's graph representation, and incorporates an adaptive re-weighting module to deal with the long-tail distribution in the sample sizes across diverse vulnerability types.
- **UniXcoder_{fft}** and **UniXcoder_{lora}** are both built upon UniXcoder (Guo et al. 2022), a unified cross-schema pre-trained model for programming languages. These two models are learned by taking structurally-aware token sequences as input to adapt UniXcoder for vulnerability detection, via the typical full fine-tuning and LoRA fine-tuning, respectively, to compare against our adversarial reprogramming approach.
- **DeepSeek-V3.1** baseline refers to directly prompting DeepSeek-V3.1, one of the most advanced LLMs, for vulnerability detection. The prompt design follows the strategy proposed in Guo et al. (2024).

4.1.4 Evaluation metrics

Consistent with most existing DL-based vulnerability detection methods, we use Accuracy (Acc.), Precision (Prec.), Recall (Rec.), and F1 score as primary metrics

for fair performance comparison. Additionally, in Section 4.2.6, we employ False Positive Rate (FPR) and False Negative Rate (FNR), as used in the ZigZag work (Li et al. 2021a), to evaluate model robustness against potential code transformations and facilitate comparison with the ZigZag-enabled models. Specifically, these metrics are formulated as follows:

$$Acc = \frac{TP + TN}{TP + FP + TN + FN} \quad (13)$$

$$Prec = \frac{TP}{TP + FP}, \quad Rec = \frac{TP}{TP + FN} \quad (14)$$

$$F1 = 2 \times \frac{Prec \times Rec}{Prec + Rec} \quad (15)$$

$$FPR = \frac{FP}{FP + TN}, \quad FNR = \frac{FN}{TP + FN} \quad (16)$$

where TP denotes the number of vulnerable samples correctly detected by the model, FP is the number of benign samples incorrectly classified as vulnerable, TN denotes the number of benign samples correctly predicted as benign, and FN is the number of vulnerable samples not detected by the model.

4.2 Experimental results

4.2.1 RQ1: Performance on vulnerability detection

In this experiment, we compare ArVD with the baselines across four datasets (TrVD, Reveal, Devign and BigVul) to assess vulnerability detection performance. Table 2 summarizes the results, demonstrating that ArVD outperforms the majority baseline models in most performance metrics across all datasets. Specifically, on the synthetic TrVD dataset, ArVD attains the second best accuracy of 88.24% and the highest precision compared among the baseline models, but its relatively low recall indicates that many vulnerable samples are missed. This reflects a precision–recall trade-off, suggesting that ArVD is conservative in its predictions on this dataset. On the real-world Devign dataset, HardVD achieves the best accuracy and precision, while ArVD achieves the highest F1 score (57.42%) and recall (63.33%), outperforming other models. However, the relatively low precision of ArVD indicates its tendency to over-predict vulnerabilities. That said, while ArVD learnt on this dataset is more sensitive to detecting true vulnerabilities, it does so at the cost of introducing additional false positives. In the case of Reveal dataset, where only 9.85% of samples contain vulnerabilities, baselines perform poorly in terms of F1 score; conversely, ArVD excels on the Reveal dataset, achieving an F1 score of 42.75%, which is averagely 11.94% higher than other methods. Similar results are observed on the BigVul dataset, where ArVD exhibits the highest accuracy of 94.83% and F1 score of 37.45%, outperforming other methods by average improvements of 4.6% and 12.8%, respectively.

Table 2 Comparison against the baseline models (%)

Method	TrVD				Devign			
	Acc.	F1	Prec.	Rec.	Acc.	F1	Prec.	Rec.
TokenLSTM	83.12	82.68	81.14	84.28	53.26	41.72	45.52	38.50
TokenCNN	84.56	80.15	76.52	84.14	54.25	45.27	47.13	43.56
VulCNN	88.03	87.21	88.22	86.22	56.17	48.87	52.49	45.71
Reveal	86.29	79.34	85.67	73.88	54.37	43.48	47.43	40.14
UniXcoder _{fft}	88.22	81.58	96.94	70.42	57.43	54.96	49.53	61.72
TrVD	87.88	83.28	85.16	81.48	56.19	49.13	52.64	46.06
LineVul	88.12	82.87	88.92	77.60	59.66	54.24	53.48	55.01
EPVD	87.84	82.67	87.61	78.25	59.63	53.40	53.56	53.24
HardVD	89.21	87.91	87.91	87.91	64.21	56.29	58.02	54.65
CAPTURE	88.06	81.67	96.75	70.59	58.57	53.53	52.20	54.93
LIVABLE	84.04	81.60	84.24	80.24	61.11	56.14	58.76	53.75
UniXcoder _{lora}	88.03	81.12	77.94	84.57	57.03	54.82	52.52	57.33
DeepSeek-V3.1	58.35	48.59	44.97	52.85	54.87	34.52	51.75	25.90
ArVD	88.24	81.41	98.27	69.49	60.48	57.42	52.52	63.33
Method	Reveal				BigVul			
	Acc.	F1	Prec.	Rec.	Acc.	F1	Prec.	Rec.
TokenLSTM	86.10	26.51	25.91	27.14	88.39	25.81	20.68	34.32
TokenCNN	87.42	24.34	21.90	27.38	90.17	26.86	23.88	30.68
VulCNN	87.75	33.60	48.64	25.67	80.60	26.38	55.85	17.27
Reveal	89.07	26.97	30.00	24.49	80.44	20.72	13.37	46.04
UniXcoder _{fft}	90.14	33.44	39.23	29.14	94.42	24.56	55.16	15.8
TrVD	83.47	32.37	27.61	39.13	94.37	10.65	58.46	5.86
LineVul	87.68	30.35	31.77	29.05	94.48	22.19	55.56	13.86
EPVD	87.68	37.22	35.17	39.52	94.76	29.17	57.33	19.56
HardVD	88.73	37.84	46.67	31.82	94.58	22.22	40.00	15.38
CAPTURE	86.32	30.11	28.51	31.90	94.74	31.36	62.86	20.89
LIVABLE	87.56	39.32	36.82	42.19	94.56	37.23	44.38	32.06
UniXcoder _{lora}	85.34	35.87	26.77	54.35	90.60	25.56	23.50	28.02
DeepSeek-V3.1	69.26	18.05	12.36	33.48	80.84	17.29	11.51	34.76
ArVD	89.40	42.75	41.05	44.60	94.83	37.45	58.76	27.48

It can be observed that no single model consistently dominates across all datasets and evaluation metrics. Nevertheless, ArVD delivers competitive performance, particularly on the real-world datasets. Among three of the four evaluated datasets, ArVD achieves the highest F1 scores, which are generally believed to provide a more reliable indicator of detection effectiveness (Zhang et al. 2023; Li et al. 2021b; Das et al. 2025; Risse et al. 2025), especially in the presence of highly imbalanced class distributions, as in Reveal and BigVul.

Answer to RQ1: ArVD exhibits remarkable performance among the assessed DL models, particularly on real-world datasets, demonstrating high F1 score. Nonetheless, ensuring reliable detection of real-world programs remains a significant challenge. DL models require further enhancements to achieve more satisfactory detection performance.

4.2.2 RQ2: Efficiency analysis

In this section, we analyze the efficiency of ArVD, which undergoes data processing and adversarial reprogramming model training. In the data processing step, functions are extracted and standardized, and then parsed into sequences by incorporating structural information, which transforms the code into a format understandable by the neural network for subsequent training and prediction. In the adversarial reprogramming step, vulnerability detection is achieved by optimizing the perturbed elements through gradient updates. To conduct efficiency analysis, we input each sample into ArVD for detection and record the time consumed for data processing and adversarial reprogramming model training. The results are illustrated in Fig. 6. On average, ArVD spends 3 milliseconds (ms) on data processing for each sample, while the adversarial reprogramming model consumes 7.8ms for training. In terms of GPU memory consumption, ArVD requires approximately 485MB for model loading. Its training memory usage scales linearly with the batch size, consuming approximately 9,700MB for a batch size of 16, comparable to the fine-tuned UniXcoder_{fft} and UniXcoder_{lora} that take about 9,700MB and 7300MB, respectively.

To further demonstrate the efficiency superiority of the ArVD model, we compare it with the other models on the TrVD dataset. The average time costs of these models are reported in Fig. 6. As observed, TokenBiLSTM and TokenCNN are the most efficient as they only perform simple lexical analysis to obtain sequences and utilize CNN and BiLSTM models for vulnerability detection, which require only 1ms and about 3ms for data processing and model training, respectively. The time costs of TrVD are 5ms and 648ms, respectively. The low efficiency is due to its recursive behavior in visiting the nodes and aggregating their feature vectors hierarchically following the tree structure. Reveal, based on graphs, has time costs of 102ms and 27.7ms, primarily due to CPG construction and feature extraction using GGNN, which are computationally intensive. VulCNN's time is majorly spent on data processing. It generates PDGs and calculates centrality measures, which are then mapped to three channels of an image and then input to a CNN for vulnerability detection. Thus, VulCNN has time costs of 575ms and 0.7ms. The time costs for fine-tuning UniXcoder for vulnerability detection are 3ms and 25.2ms, respectively. Compared to UniXcoder_{fft} and UniXcoder_{lora}, our proposed model ArVD is approximately 3.2 times and 2 times faster in training speed, respectively.

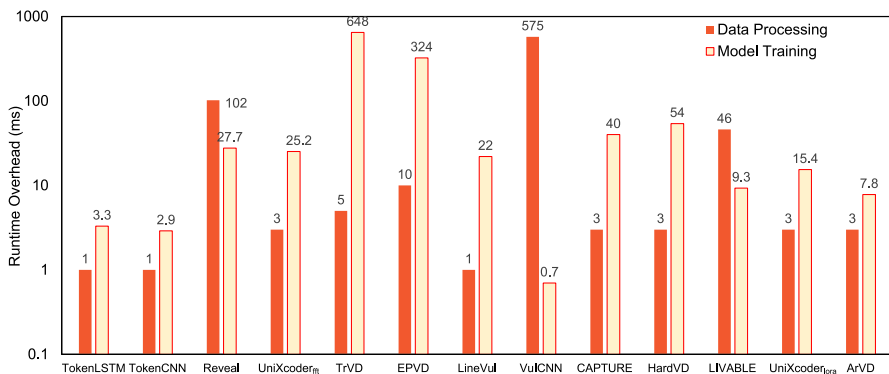


Fig. 6 Runtime overhead averaged on a single sample

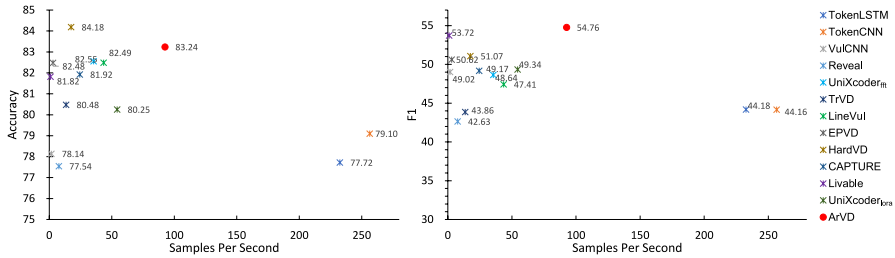


Fig. 7 Comparison of processing speed and performance

To gain better insight into the efficiency of ArVD, we compare different models on the two real-world datasets by considering both their detection performance and efficiency. As illustrated in Fig. 7, while TokenLSTM and TokenCNN exhibit the fastest processing speed, they suffer from the reduced detection effectiveness. Conversely, Reveal performs poorly in both efficiency and performance, achieving averagely only 77.54% accuracy and 42.63% F1 score. TrVD excels in data processing compared to UniXcoder, as it operates directly on the AST, whereas UniXcoder necessitates additional AST traversal to obtain the structure-aware token sequence. However, its training efficiency significantly lags behind UniXcoder due to its use of a tree neural network. LineVul and UniXcoder, which are fine-tuning based methods using pre-trained BERT models, outperform Reveal, and VulCNN in terms of efficiency because they avoid complex data processing required by those methods. However, they do not match the accuracy and efficiency of ArVD. In comparison to the two cross-modal adversarial reprogramming methods, CAPTURE and HardVD, ArVD achieves competitive or slightly superior detection performance, while offering substantially improved training efficiency, being approximately 3.8 times faster than CAPTURE and 5.3 times faster than HardVD. Overall, ArVD strikes a balance between efficiency and accuracy, outperforming other models in this regard, reaffirming its benefits in real-world vulnerability detection.

While improving training efficiency is a primary motivation for designing ArVD, inference efficiency is also a critical factor for practical deployment of vulnerability detectors. To evaluate this aspect, we further measure the inference time by randomly selecting 1,000 samples from the BigVul dataset and processing them sequentially with each trained model. As shown in Fig. 8, the average per-

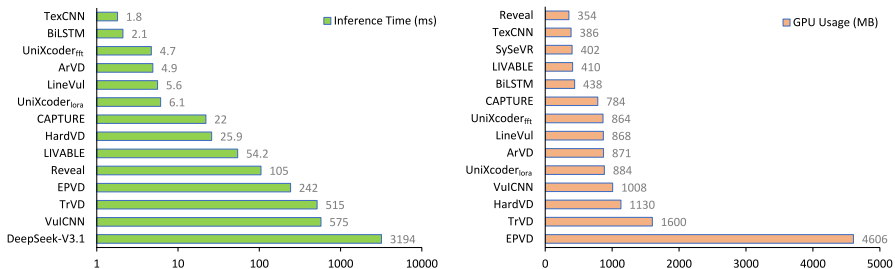


Fig. 8 Comparison of inference speed and GPU consumption

sample inference time (from raw code input to predicted label output) demonstrates that ArVD achieves competitive inference efficiency, ranking among the top four of the 14 evaluated models. Regarding GPU memory usage during inference, ArVD requires approximately 871 MB, which is comparable to other PLM-based methods and places it in the mid-range among the evaluated models.

Answer to RQ2: By freezing the pretrained backbone and substantially reducing the number of trainable parameters, ArVD achieves effective vulnerability detection while offering higher training and inference efficiency and moderate GPU memory usage compared to other DL-based baselines.

4.2.3 RQ3: Data-limited training

In this experiment, we evaluate the performance of ArVD under data-limited conditions. While DL models have demonstrated remarkable performance in vulnerability detection tasks, their effectiveness heavily relies on access to a substantial number of labeled instances during training. Unfortunately, practical applications often struggle to obtain a sufficient quantity of labeled training samples for vulnerabilities due to constraints in data volume and sample acquisition. The resulting imbalance and scarcity of samples can detrimentally impact model performance. Specifically, we select a subset of samples from the TrVD, Reveal, Devign and BigVul datasets for this experiment. Each time, we randomly extract 10, 50, 100, 500, and 1,000 samples from the training set to perform data-limited training, while keeping the validation and test sets unchanged. Across 10 runs of training and testing, we compute the average results to ensure reliability and consistency in our analysis.

Figure 9 presents the experimental results, highlighting two important metrics: accuracy and F1 score. On the TrVD dataset, ArVD achieves a significant accuracy lead, reaching 86.46% with only 1,000 training samples, outperforming TokenLSTM, TokenCNN, and Reveal models that rely on the full dataset. It also surpasses the other models with an average of 8.44% absolute accuracy improvement and a 17.97% increase in F1 score. This achievement is attributed to leveraging pre-trained model knowledge and the stability enhancement provided by the adversarial reprogramming approach, facilitating high performance despite limited sample sizes. On the Devign dataset, HardVD emerges as the top-performing model on accuracy. Although ArVD does not match HardVD on this dataset, it still outperforms the rest models, with an average margin of 2.77% in accuracy and 3.72% in F1 score. Regarding the imbalanced Reveal dataset, ArVD achieves the highest F1 score across all five data-limited scenarios. However, TokenLSTM and TokenCNN achieve the highest accuracy albeit with lower F1 scores due to the fact that they tend to predict the majority classes more frequently, resulting in elevated accuracy but potentially poorer performance on minority classes. Similar results are observed on the BigVul dataset, where ArVD generally outperforms the other baseline models, with average performance gains of 13.79%

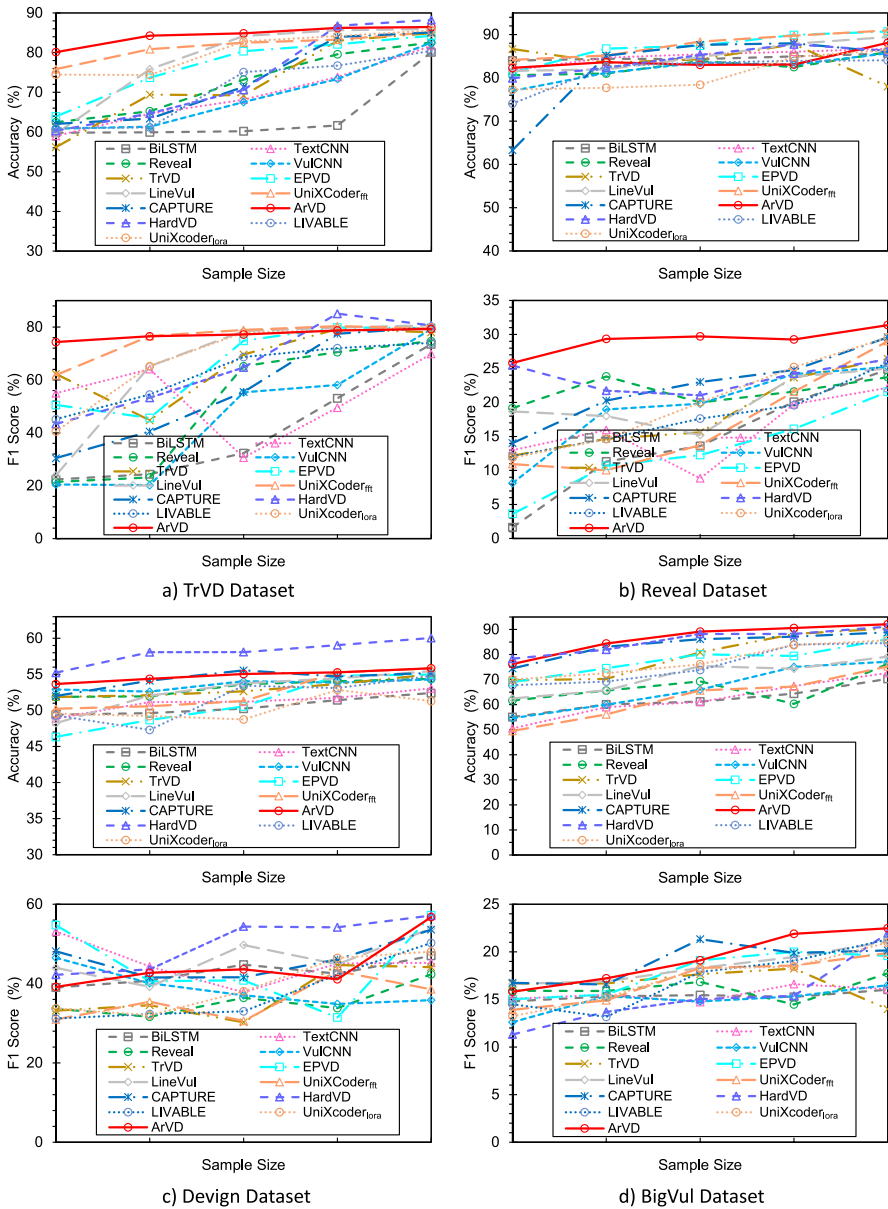


Fig. 9 Comparison of different models in the case of data-limited training

in accuracy and 2.56% in F1 across the data-limited scenarios. Overall, ArVD demonstrates competitiveness among other DL models and delivers outstanding performance even with limited data.

Answer to RQ3: ArVD exhibits impressive performance in data-limited scenarios, achieving significant results with relatively small datasets. It even surpasses other baseline methods on the entire dataset.

4.2.4 RQ4: Impact of perturbation element number and pre-trained model

In the previous experiments, ArVD is configured with the optimal settings: 27 perturbation elements and UniXcoder as the pre-trained model. In this experiment, we detail the process of determining these optimal settings by exploring how different numbers of perturbation elements and various pre-trained models affect the effectiveness of ArVD.

Specifically, we replace UniXcoder with several widely-used alternatives as the pre-trained model for ArVD, including RoBERTa (Liu et al. 2019), CodeBERT (Feng et al. 2020), and GraphCodeBERT (Guo et al. 2020), whose architectural details are summarized in Table 3. For each variant model, its detection performance is assessed with different numbers of perturbation elements, starting from 0, which indicates no added perturbations but involves learning a linear layer for label remapping. Subsequently, we experiment with 10, 21, 23, 25, 27, 29, and 40 perturbation elements across the different datasets.

The summary of experimental results is illustrated in Fig. 10. It is evident that selecting 0 perturbation elements generally yields the poorest performance across the datasets. This is primarily because the pre-trained models are not fine-tuned in the absence of perturbations, while featuring only one learnable linear layer for label remapping during training may result in weak generalization ability for the new task. As we increase the number of learnable perturbation elements, the models' performance tends to improve as well. We also observe that the overall performance peaks with 27 perturbation elements. However, further increasing the number of perturbation elements has a limited impact on enhancing model performance and may even result in worse outcomes compared to using fewer perturbation elements. Increasing the number of perturbation elements may increase the complexity of the model, but indiscriminately adding more elements may result in performance degradation. Additionally, increasing the number of perturbation elements adds complexity to the model and requires more labeled data and computational resources due to the greater number of learnable parameters.

Table 3 Pre-trained model architecture details

Architecture	RoBERTa	CodeBERT	UniXcoder	GraphCodeBERT
Parameters	125M	125M	125M	125M
Pretrain on Code	×	✓	✓	✓
Pretrain on C/C++	×	×	×	×
Max Length	512	512	1,024	512

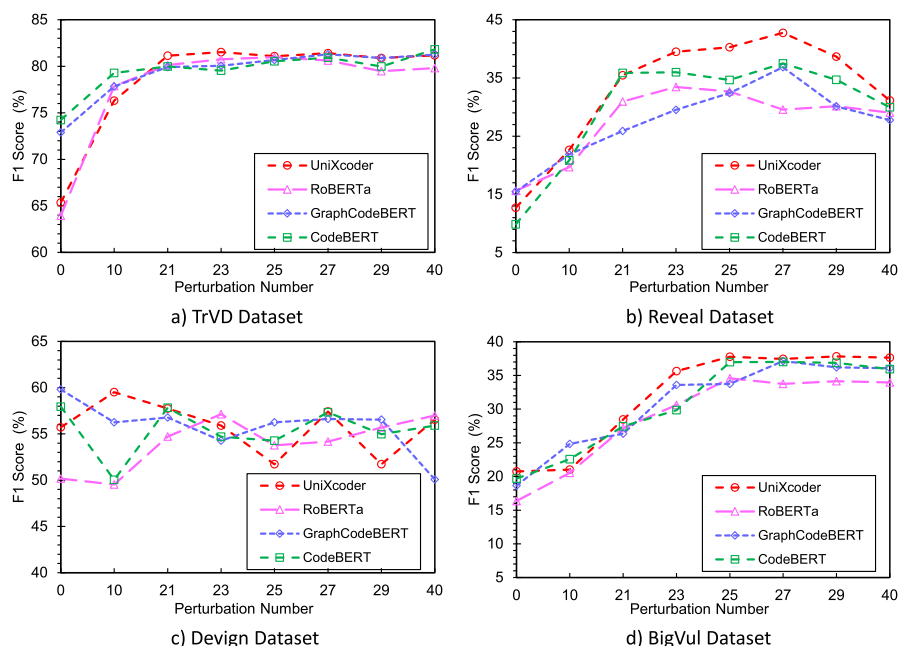


Fig. 10 Impact of numbers of perturbation elements and pre-trained models

Among all the pre-trained models, UniXcoder is the most compatible with ArVD, achieving improvements of 3.36% in F1 score when 27 perturbation elements are inserted, compared to other pre-trained models. The reason behind its superior performance is that UniXcoder has an input sequence length of 1,024 that accommodates more tokens and thus enhances the model's comprehension of code. While ArVD reprogrammed on CodeBERT and GraphCodeBERT show relatively lower performance compared to UniXcoder, these models still outperform their fine-tuning models with higher overall F1-scores on the datasets. Moreover, their efficiency improves significantly when employing the adversarial reprogramming method compared to model fine-tuning. Despite not being directly trained on code, RoBERTa also proves effective in vulnerability detection tasks, which demonstrates the feasibility and effectiveness of our adversarial reprogramming approach to adapt pre-trained models for new tasks. Overall, ArVD reprogrammed with UniXcoder and 27 perturbation elements emerges as the strongest option across the four tested datasets, and this configuration is used for evaluation and comparison throughout the experiments.

Answer to RQ4: As the number of perturbation elements added to the inputs increases, the performance of ArVD generally improves. However, once a certain threshold is reached, further increases in perturbation elements do not enhance performance and may even lead to deterioration. Also, ArVD has demonstrated its effectiveness with different pre-trained language models. The adoption of adversarial reprogramming in ArVD allows the original pre-trained models to competently perform vulnerability detection tasks.

4.2.5 RQ5: Impact of perturbation position

By default, ArVD employs a random strategy for placing perturbation elements within the input sequence. In addition to this random placement, we further examine ArVD's detection performance under three alternative strategies: front- k , middle- k , and tail- k . These strategies insert k consecutive perturbation elements at fixed positions $[1, k]$, $[\lfloor n/2 \rfloor, \lfloor n/2 \rfloor + k]$, and $[n - k + 1, n]$ of the input, respectively.

As reported in Table 4, when reprogrammed on UniXcoder with $k = 27$ perturbation elements, ArVD exhibits only minor performance differences of 0.06%–0.66% in accuracy and 0.04%–1.41% in F1 score across the datasets under the different placement strategies. These marginal differences suggest that ArVD is largely insensitive to perturbation positions, consistently maintaining stable detection performance regardless of placement.

Answer to RQ5: ArVD demonstrates only minor performance fluctuations across the tested perturbation placement strategies, indicating that it is largely insensitive to the choice of perturbation positions.

4.2.6 RQ6: Robustness evaluation

In this experiment, we compare ArVD with DL-based vulnerability detection models enabled by the state-of-the-art ZigZag framework (Li et al. 2021a) to answer RQ5. The ZigZag framework enhances the robustness of DL-based vulnerability detectors against semantics-preserving code transformation attacks, by iteratively optimizing a feature generator and two classifiers. The feature generator produces features capable of effective classification across diverse decision boundaries, while the two classifiers are trained to be complementary and mutually contradictory, thereby increasing resistance against adversarial examples. By enforcing ZigZag training on existing detectors, their robustness can thus be improved particularly when confronted with attacks involving code transformations (Li et al. 2021a).

Table 4 Performance under different perturbation insertion location (%)

Position	TrVD				Devign			
	Acc.	F1	Prec.	Rec.	Acc.	F1	Prec.	Rec.
Random	88.24	81.41	98.27	69.49	60.48	57.42	52.52	63.33
Front	88.72	82.20	97.48	71.06	59.03	56.21	51.58	61.76
Middle	88.17	81.58	96.48	70.66	60.03	56.25	52.55	60.52
Tail	88.08	81.18	97.80	69.39	59.34	55.84	50.40	62.61
Variance	0.06	0.14	0.43	0.53	0.32	0.35	0.77	1.09
Position	Reveal				BigVul			
	Acc.	F1	Prec.	Rec.	Acc.	F1	Prec.	Rec.
Random	89.40	42.75	41.05	44.6	94.83	37.45	58.76	27.48
Front	89.13	43.02	43.83	42.23	94.98	38.25	60.78	27.91
Middle	88.72	43.31	44.73	41.97	93.38	36.14	58.12	26.22
Tail	89.87	42.87	43.37	42.39	93.19	35.16	57.63	25.3
Variance	0.17	0.04	1.85	1.11	0.66	1.41	1.44	1.06

To facilitate robustness evaluation and ensure fair comparison with ZigZag, we employ the same eight semantic-preserving code transformations supported by Tigress to derive counter attack samples. A brief introduction of these transformations is as follows (Li et al. 2021a):

- **CT1:** Substitute the literal strings with calls to the function that generates these literal strings.
- **CT2:** Reorder arguments and/or add bogus arguments.
- **CT3:** Remove some control flows from a function.
- **CT4:** Merge functions without control-flow flattening.
- **CT5:** Merge functions with control-flow flattening.
- **CT6:** Split top-level statements into multiple functions.
- **CT7:** Split a basic block into multiple functions.
- **CT8:** Split a basic block into multiple functions, and split the calls to split functions.

We apply these code transformation attacks to the original programs to generate the manipulated versions. During this process, we randomly split 6,803 programs from the ZigZag dataset, allocating 80% for training and 20% for testing. CT-2, CT-7, and CT-8 serve as known code transformations that allow the defender to extend the training programs into a larger set that includes both original and transformed programs, resulting in 26,181 vulnerable examples and 40,994 non-vulnerable examples. The testing programs comprise the original programs along with their manipulated programs using all 8 code transformations, yielding 17,516 vulnerable examples and 28,206 non-vulnerable examples.

Under the same experimental setting as above, we compare ArVD with ZigZag-SySeVR, ZigZag-CNN, and ZigZag-BiLSTM, which are the enhanced versions of SySeVR (Li et al. 2021c), TokenCNN, and TokenLSTM learned using the ZigZag framework, respectively. Also, we assess the robustness of other comparison methods, including LineVul, UniXcoder_{fft}, Reveal, EPVD, VulCNN, TrVD, CATPURE, HardVD, LIVABLE, UniXcoder_{lora}, by adversarially training them on the augmented training set that incorporates both the original samples and their code transformation manipulated variants. In addition, DeepSeek-V3.1 is directly prompted to assess its resilience against the semantics-preserving code transformation attacks.

Table 5 summarizes the experimental results. In comparison to the three ZigZag-enabled detectors, ArVD exhibits an average relative increase of 18.89% in F1 score, a decrease of 43.25% in FPR, and a decrease of 42.65% in FNR. This confirms that the adversarial reprogramming model ArVD is capable of effectively countering code transformation attacks, demonstrating superior robustness compared to the state-of-the-art ZigZag framework, which is specifically designed to confront semantics-preserving code transformation attacks. Besides, the significantly lower metric values of the other compared methods in Table 5 indicate their effectiveness is largely compromised by these code transformation attacks, even when adversarially trained on augmented data derived from the original samples with semantics-preserving code transformation. Notably, ArVD significantly outperforms them in this scenario, with an average relative improvement of 25.95% in F1 score, a decrease of 51.90% in FPR,

Table 5 Robustness evaluation between ArVD and other vulnerability detectors against code transformation attacks (%)

Attacks	ZigZag-SySeVR			ZigZag-CNN			ZigZag-BiLSTM			ArVD		
	F1↑	FPR↓	FNR↓	F1↑	FPR↓	FNR↓	F1↑	FPR↓	FNR↓	F1↑	FPR↓	FNR↓
CT-2	85.2	6.4	15.7	72.3	15.2	30.4	76.8	19.3	19.5	89.69	4.74	14.07
CT-7	81.2	8.7	19.6	66.2	22.1	33.9	74.5	22.6	20.1	89.51	8.04	11.27
CT-8	81.2	8.9	19.5	64.5	23.0	35.7	73.9	20.2	23.2	89.21	8.89	10.88
CT-1	80.3	9.8	19.1	70.9	23.5	25.1	74.6	21.3	20.9	85.69	9.73	16.54
CT-3	79.5	9.7	21.2	70.2	23.3	26.7	73.9	14.9	28.1	84.75	9.16	18.02
CT-4	79.5	9.4	21.7	63.1	32.4	34.5	72.3	19.5	29.2	87.22	8.96	11.84
CT-5	78.8	10.3	21.6	62.1	33.4	35.5	71.4	20.8	29.5	84.62	9.40	15.77
CT-6	79.4	9.5	21.7	66.8	22.6	31.7	74.2	19.6	22.5	87.88	8.71	13.51
Average	80.64	9.09	20.01	67.01	24.44	31.69	73.95	19.78	24.13	87.32	8.45	13.99
Attacks	LineVul			UniXcoder _{fft}			Reveal			EPVD		
	F1↑	FPR↓	FNR↓	F1↑	FPR↓	FNR↓	F1↑	FPR↓	FNR↓	F1↑	FPR↓	FNR↓
	73.01	18.35	24.18	77.45	19.76	22.28	71.89	21.56	18.66	86.26	8.20	16.70
	70.53	18.19	28.61	73.75	15.44	26.17	73.77	17.70	29.12	87.15	8.88	14.64
	75.65	18.00	26.00	74.57	14.23	25.61	73.89	18.27	24.36	85.60	11.64	15.04
	67.73	24.52	27.11	68.03	23.57	27.46	67.15	19.09	34.10	82.39	10.19	21.37
	70.18	15.73	30.95	66.41	19.23	33.31	70.55	24.86	29.13	80.37	11.58	20.67
	66.73	18.64	31.08	67.98	20.77	26.91	70.02	20.89	32.54	83.52	11.23	18.74
	62.82	16.90	40.84	66.82	22.57	27.05	70.16	18.41	32.03	81.95	10.05	22.32
	70.33	23.26	23.64	71.69	18.52	25.97	69.31	21.86	33.02	80.76	17.13	18.88
	69.62	19.20	29.05	70.84	19.26	26.85	70.84	20.33	29.12	83.50	11.11	18.55
	VulCNN			TrVD			CAPTURE			HardVD		
F1↑	FPR↓	FNR↓	F1↑	FPR↓	FNR↓	F1↑	FPR↓	FNR↓	F1↑	FPR↓	FNR↓	
CT-2	70.05	15.08	36.31	85.23	12.11	14.95	79.68	17.90	18.98	75.13	18.87	27.55
CT-7	69.43	15.88	36.97	85.54	10.28	14.42	81.37	16.57	17.23	78.57	10.53	26.67
CT-8	67.62	15.10	39.20	82.56	10.23	17.44	77.31	15.07	25.57	75.18	9.70	28.57
CT-1	63.53	13.95	45.72	81.74	10.19	25.20	75.83	18.82	20.78	73.61	11.40	33.33

Table 5 (continued)

Attacks	ZigZag-SySeVR				ZigZag-CNN				ZigZag-BiLSTM				ArVD	
	F1↑	FPR↓	FNR↓	F1↑	FPR↓	FNR↓	F1↑	FPR↓	F1↑	FPR↓	FNR↓	F1↑	FPR↓	FNR↓
CT-3	69.22	17.42	35.46	77.10	10.17	27.62	81.36	13.40	19.91	13.76	32.63	73.56	13.76	32.63
CT-4	62.99	17.11	41.62	76.44	10.24	28.55	77.36	18.23	18.94	13.37	34.25	72.41	13.37	34.25
CT-5	63.19	15.32	42.54	81.79	13.29	16.30	75.83	18.82	20.78	15.82	33.99	69.13	15.82	33.99
CT-6	64.68	15.53	43.35	80.06	13.10	25.49	79.28	17.51	20.67	38.83	43.56	70.37	38.83	43.56
Average	66.34	15.67	40.15	81.31	11.20	21.25	78.50	17.04	20.36	16.54	32.57	73.50	16.54	32.57
Attacks	LIVABLE				UniXcoder _{torra}				DeepSeek-V3.1					
	F1↑	FPR↓	FNR↓	F1↑	FPR↓	FNR↓	F1↑	FPR↓	FNR↓	F1↑	FNR↓	F1↑	FPR↓	FNR↓
CT-2	83.58	10.76	14.82	73.82	23.93	17.24	45.67	63.01	38.10	43.30	35.59	43.33	41.75	40.48
CT-7	77.33	18.28	17.27	73.39	23.39	18.87	43.54	60.73	43.30	35.59	43.33	41.75	40.48	43.67
CT-8	73.02	24.11	18.01	70.75	20.78	25.18	46.22	64.75	35.59	43.33	41.75	40.48	43.67	45.81
CT-1	74.19	22.53	18.08	68.41	22.52	27.78	43.63	59.67	43.33	41.75	40.48	43.67	45.81	41.50
CT-3	75.13	22.28	16.03	68.22	22.16	27.86	43.29	62.42	41.75	40.48	43.67	45.81	41.50	41.50
CT-4	73.45	19.31	19.33	66.24	22.33	28.15	43.32	57.09	40.48	43.67	45.81	41.50	41.50	41.50
CT-5	73.08	22.65	16.20	67.22	21.33	27.66	42.50	54.06	43.67	45.81	41.50	41.50	41.50	41.50
CT-6	76.48	23.01	13.09	71.24	21.44	23.86	41.59	60.62	45.81	41.50	41.50	41.50	41.50	41.50
Average	75.78	20.37	16.60	69.91	22.24	24.58	43.72	60.29	41.50	41.50	41.50	41.50	41.50	41.50

Note: The data for ZigZag-SySeVR, ZigZag-CNN and ZigZag-BiLSTM are quoted from the ZigZag (Li et al. 2021a)

and a decrease of 44.46% in FNR. When compared to CAPTURE and HardVD, ArVD outperforms them with a clear 11.24% and 18.80% relative increase in F1, a 50.41% and 48.91% relative reduction in FPR, and a 31.29% and 57.05% relative reduction in FNR, respectively. This suggests the limitations of their adopted perturbation injection strategies in reprogramming the model to deal with the high variability introduced by code transformations. Moreover, as reflected in the CT-2, CT-7, and CT-8 rows (where models are adversarially trained with transformation-augmented data and evaluated on samples containing the same type of transformation), the performance is generally higher than in other CT-x rows, where no augmented data of these CT-x types are observed during training. This indicates that incorporating adversarial samples can enhance the robustness of the learned model; however, its effectiveness diminishes on code samples processed with transformation strategies not encountered during training.

Answer to RQ6: The perturbation elements act on the token sequences through a non-linear self-attention operation, regularizing ArVD's model optimization and thus enhancing its robustness against adversarial code transformation attacks. Also, ArVD's robustness surpasses that of existing vulnerability detectors with significant margins, even when those models are adversarially trained with code transformation manipulated samples.

5 Discussion

5.1 Threats to validity

It is essential to acknowledge that the functions in the established public vulnerability datasets are often isolated and generally not syntactically correct or compilable, due to missing reliant external variables, function calls, macro definitions, and other elements. In our implementation, we employ tree-sitter to generate ASTs. Although widely adopted in the literature, tree-sitter might sometimes fail to parse a code sample accurately, thus impacting the vulnerability detection capability of the model. However, similar to existing vulnerability detectors, we consider it an essential ability of the detection model in identifying vulnerability patterns implied within the code, regardless of its syntactic correctness, as developers often submit incomplete code snippets for vulnerability analysis and detection in real-world scenarios. That said, we believe more advanced parsers could further help improve the model's vulnerability detection capabilities.

In evaluating the robustness of ArVD, we primarily focus on semantics-preserving code transformation attacks. Besides the eight code transformation attacks we have evaluated against, other semantics-preserving transformations exist, such as identifier renaming (Zhang et al. 2020; Yefet et al. 2020) and dead code insertion (Yefet et al. 2020). For the identifier renaming attack, it poses no threat to our method, as our normalization process eliminates its impact before the input is fed into the trained model for detection. For dead code insertion attack, simple strategies like those used by DAMP (Yefet et al. 2020), which only insert a single variable declaration statement at the end of the target code, should have minimal impact on the detection

performance of DL-based detectors. However, more complex dead code insertion strategies, such as inserting large amounts of benign dead code into vulnerable samples or inserting substantial dead code containing certain vulnerabilities into benign samples, can potentially deceive DL-based vulnerability detectors, leading to incorrect predictions.

Statistical significance testing is relatively uncommon in deep learning, as drawing reliable conclusions typically requires a sufficient number of runs (e.g., 10 or more) (Rainio et al. 2024), whereas training DL models generally incurs substantial computational cost. In line with most prior DL-based vulnerability detection studies (Li et al. 2021c; Chakraborty et al. 2021; Fu and Tantithamthavorn 2022; Zhang et al. 2023), we did not perform significance testing between ArVD and other methods, due to the considerable training efforts and lengthy runtimes required for some baselines such as TrVD and EPVD. This limitation may affect the statistical robustness of our comparisons.

5.2 Limitations and future works

Different from rule-based vulnerability detection models that furnish detection results with extra information to indicate potential bug-triggering paths of identified vulnerabilities (Cheng et al. 2024), ArVD, as a DL-based approach, is confined to offering black-box detection results regarding the presence of vulnerabilities in given code, without providing explanations for the predictions. Several studies (Li et al. 2021b; Cheng et al. 2024, 2022) have investigated the application of explainable AI (XAI) or fine-grained embedding techniques to highlight the statements or paths that substantially contribute to prediction outcomes. We leave the exploration of applying adversarial reprogramming and XAI techniques for pinpointing vulnerable statements as interesting future works.

This study deliberately employs RoBERTa-style, encoder-only pre-trained language models, such as UniXcoder and CodeBERT, as the backbone for ArVD. At present, our adversarial reprogramming paradigm has not been extended to encoder-decoder or decoder-only architectures. Moving to such models (e.g., NatGen (Chakraborty et al. 2022) or GPT-based LLMs) would necessitate substantial methodological and implementation changes. We therefore acknowledge this as a current limitation and identify the integration of more sophisticated PLMs as a promising avenue to further improve ArVD's detection performance and robustness. We intend to explore these extensions in future work.

Current DL-based methods and the pre-training models face limitations regarding sequence length. For instance, the maximum input sequence length for SySeVR is 500 tokens, while for CodeBERT, it is 512. Such constraints could result in overlooking potential vulnerability information that resides after the truncation point of the code being analyzed. In our future research, we plan to explore methods for increasing the encoding capacity of adversarial reprogramming techniques, such as firstly converting the code into more compact representations with lightweight neural encoding, and then applying adversarial reprogramming on pre-trained models to comprehensively capture potential weaknesses in code snippets.

Two core design philosophies of ArVD are to enhance training efficiency and adapt to scenarios with limited training data. This necessitates restricting the number of trainable parameters, leading us to choose a linear classification layer. Replacing this module with more complex alternatives, such as CNN/BiLSTM+MLP or further incorporating attention mechanisms, could potentially improve ArVD's performance. However, more complex models entail more parameters to update during training. Optimizing these larger parameter sets typically requires more training data and leads to increased training time and effort. We leave the exploration of alternative structures for the label remapping module and their impacts on ArVD's detection performance, training efficiency, and robustness as a future work.

Currently, ArVD is implemented on AST-based representations rather than more complex heterogeneous graphs such as code property graph (CPG). We chose ASTs for three main reasons. First, our approach is designed around pre-trained sequence models (e.g., UniXcoder) and an adversarial reprogramming mechanism that injects perturbation elements into token sequences. AST-derived sequences naturally align with this design, while applying perturbations to heterogeneous graphs would require altering graph topology or node features, which risks semantic drift and complicates training. Second, the construction of heterogeneous code graphs such as CPGs is less mature and more error-prone than AST construction, particularly when processing incomplete snippets frequently found in real-world datasets like BigVul. Third, building and processing heterogeneous code graphs typically demands time-consuming control- and data-dependency analyses, whereas ASTs can be generated and processed more efficiently, thereby improving the overall scalability of ArVD. Nevertheless, we think it promising to extend the reprogramming paradigm to graph-based encoders by perturbing the node or edge embeddings, which we identify as a promising direction for future work.

6 Related work

Recent SOTA vulnerability detection methods have predominantly leveraged DL techniques, which we discuss from the perspective of their major motivations.

6.1 Researches pursuing improved detection performance

The majority of existing DL-based methods generally focus on improving the vulnerability identification capability.

Previous works (Russell et al. 2018) primarily leverage CNN or LSTM to capture significant features from the lexically parsed code token sequences. As demonstrated in our experimental results with Baseline-BiLSTM and Baseline-TextCNN, these approaches offer the advantage of lightweight code processing, but may overlook semantic dependencies crucial for vulnerability prediction. VulBERTa (Hanif and Maffeis 2022) also operates on the token sequences, but conducts continued pre-training and fine-tuning on the standard RoBERTa model for better detection performance, which, however, incurs a significantly higher training cost than our reprogramming-based model due to the large parameters to update.

Another mainstream methods are slice-based. In this line of work (Li et al. 2018, 2021c; Zheng et al. 2024; Tang et al. 2022), the programs are sliced into small pieces based on potential vulnerability key points such as library/API calls. For instance, SySeVR (Li et al. 2021c) slices the code using vulnerability syntax features (e.g., array usage, pointer operations, and arithmetic expressions); BiGRU is then employed to predict vulnerability taking in the sliced code snippet. However, these methods exhibits lower efficiency and may overlook vulnerabilities that are unrelated to the vulnerability hotspots.

The more recent works (Chakraborty et al. 2021; Cheng et al. 2021; Jia et al. 2025) are graph-based ones. They utilize CPG (Zhou et al. 2019), CFG (Cao et al. 2021), PDG (Li et al. 2021b) and elaborately derived structures (Wen et al. 2023), such as XFG (Cheng et al. 2021) and EFG (Wang et al. 2023) as code representations, and leverage graph neural networks (GNNs) to extract higher-level code features for vulnerability prediction. For example, Reveal (Chakraborty et al. 2021) utilizes CPG to extract both syntax and semantics from the code; each node in the CPG is assigned a GRU is assigned using GGNN (Li et al. 2016), which updates the embedding of current node by performing neighborhood aggregation. However, constructing graph structures and training high-performance GNN models tends to be complex and time-consuming. They often struggle to capture global information within the code graph, such as long-range dependencies between nodes.

There are also emerging works (Ding et al. 2025; Zhou et al. 2024; Ren et al. 2024; Guo et al. 2024; Yin et al. 2024; Lu et al. 2024) that have explored the use of general-purpose LLMs, such as ChatGPT and LLaMA, for vulnerability detection by employing specially designed prompts. However, these pioneering efforts reveal that LLMs face significant challenges in effectively identifying vulnerabilities, particularly in understanding critical code structures and security-related concepts (Steenhoek et al. 2024). To address these limitations, carefully crafted prompts that integrate chain-of-thought (COT) reasoning (Nong et al. 2024) and in-context learning could potentially enhance the capability of LLMs for vulnerability detection tasks.

6.2 Researches pursuing finer granularity and interpretability

In addition to the majority of works detecting vulnerable functions, there are also works (Li et al. 2021b; Cheng et al. 2024; Cao et al. 2024; Shao et al. 2025) targeting more fine-grained statement-level detection and result interpretability. For example, IVDetect (Li et al. 2021b) utilizes GNNExplainer to highlight crucial statements in PDG for result interpretations. VELVET (Ding et al. 2022) locates the vulnerable statements by performing ensemble learning with Transformer on token sequence and GGNN on CPG. ContraFlow (Cheng et al. 2022) pinpoints the buggy paths to explain the reason for determining a function as vulnerable, through complicated contrastive value-flow embedding, active learning based path selection, and path encoding with Transformer. MVD (Cao et al. 2022) devises Flow-Sensitive GNN, which jointly embed both statement embedding and structured information (i.e., control and data flow), to achieve statement-level memory-related vulnerability detection. Our work aims for different goals of reducing the computational cost and achieving robust detection with limited data. We leave the exploration of applying adversarial reprogramming for predicting vulnerable statements as an interesting future work.

6.3 Researches for mitigating vulnerability data scarcity

The scarcity of high-quality vulnerable samples in practical settings significantly impedes the effective application of DL-based vulnerability detectors (Nong et al. 2022; Yang et al. 2023; Gao et al. 2024; Croft and Babar 2023). Consequently, methods exploring the automatic generation of vulnerable code (Nong et al. 2023b) have garnered attention to enhance the learning capabilities of these data-driven models. Nong et al. (2022) explore utilizing neural code editors, such as Graph2Edit, to derive vulnerability samples from normal programs. VulGen (Nong et al. 2023b) combines pattern mining and DL-based localization for injection-based vulnerability generation. Similarly, VGX (Nong et al. 2023a) also performs vulnerability injection to produce vulnerable samples, but incorporates data augmentation and manually derived mutation rules for code edit pattern diversification, and features a semantics-aware contextualization Transformer to identify the injection contexts.

In contrast to these vulnerability data augmentation works, HardVD (Tian et al. 2025) and CAPTURE (Tian et al. 2025), two recently proposed methods most closely related to our work, employ a cross-modal adversarial reprogramming paradigm. They convert the code into perturbation images with different strategies to repurpose a vision model for vulnerability detection, with the goal of alleviating the training efforts and the need for extensive labeled vulnerable code. While both HardVD and CAPTURE inject full-scale perturbations into the transformed representations, ArVD introduces a more parameter-efficient perturbation injection scheme and reprograms the homomodal pre-trained code models for improved data-efficient and robust vulnerability detection against semantics-preserving code transformations.

7 Conclusion

In this paper, we introduce ArVD, a novel model adversarial reprogramming based model aiming for training efficient and robust vulnerability detection. ArVD first constructs structure-aware token sequences; based on the input token sequences, it exclusively learns perturbation elements to be added to the token sequences, and leverages self-attention mechanism to enhance model nonlinearity and adapt a pre-trained model to perform vulnerability detection task with less training data and time yet higher detection effectiveness and robustness. Extensive experiments are conducted on multiple datasets, which demonstrate that ArVD outperforms DL-based baselines in terms of detection effectiveness, data-limited performance, and computational overhead, and surpasses the ZigZag framework and other baseline methods with better robustness against code transformation attacks. As our future work, we plan to extend ArVD to reprogram more advanced pre-trained models and enhance its effectiveness, robustness, and explainability.

Acknowledgements This work was supported by the National Natural Science Foundation of China (62272387, 61702414), the Youth Innovation Team Project of Science Research Program Project of Shaanxi Provincial Department of Education (25JP179), the Natural Science Basic Research of Shaanxi (2022JM-342, 2018JQ6078), and Graduate Innovation Fund of Xi'an University of Posts and Telecommunications (CXJJZL2023028).

Author Contributions Zhenzhou Tian: conceptualization, methodology. Chuang Zhang: data curation, writing-original draft, software. Yunpeng Hui: data curation, software. Jiaze Sun: investigation, methodology. Yanping Chen: validation, supervision. Lingwei Chen: methodology, writing-review and Editing. All authors reviewed the manuscript.

Data Availability The datasets used in the manuscript are all public and have been properly cited in the manuscript. The code of our proposed method are available at the GitHub repository: <https://github.com/XUPT-SSS/ArVD>.

Declarations

Competing interests The authors declare no competing interests.

References

- Cao, S., Sun, X., Bo, L., Wei, Y., Li, B.: Bgnn4vd: Constructing bidirectional graph neural-network for vulnerability detection. *Info. Sof. Tech.* **136** (2021)
- Cao, S., Sun, X., Bo, L., Wu, R., Li, B., Tao, C.: MVD: memory-related vulnerability detection based on flow-sensitive graph neural networks. In: ICSE, pp. 1456–1468 (2022)
- Cao, S., Sun, X., Wu, X., Lo, D., Bo, L., Li, B., Liu, W.: Coca: Improving and explaining graph neural network-based vulnerability detection systems. In: ICSE. ICSE '24. Association for Computing Machinery, New York, NY, USA (2024)
- Chakraborty, S., Ahmed, T., Ding, Y., Devanbu, P.T., Ray, B.: Natgen: generative pre-training by “naturalizing” source code. In: ESEC/FSE, pp. 18–30 (2022)
- Chakraborty, S., Krishna, R., Ding, Y., Ray, B.: Deep learning based vulnerability detection: Are we there yet. *IEEE Trans. Softw. Eng.* (2021)
- Chawla, N.V., Bowyer, K.W., Hall, L.O., Kegelmeyer, W.P.: Smote: synthetic minority over-sampling technique. *JAIR* **16**, 321–357 (2002)
- Chen, L., et al.: Adversarially reprogramming pretrained neural networks for data-limited and cost-efficient malware detection. In: SDM, pp. 693–701. SIAM (2022)
- Chen, L., Fan, Y., Ye, Y.: Adversarial reprogramming of pretrained neural networks for fraud detection. In: CIKM, pp. 2935–2939 (2021)
- Cheng, X., Zhang, G., Wang, H., Sui, Y.: Path-sensitive code embedding via contrastive learning for software vulnerability detection. In: ISSTA, pp. 519–531 (2022)
- Cheng, B., Zhao, M., Wang, K., Wang, M., Bai, G., Feng, R., et al.: Beyond fidelity: Explaining vulnerability localization of learning-based detectors. *ToSEM* (2024)
- Cheng, X., Wang, H., Hua, J., Xu, G., Sui, Y.: Deepwukong: Statically detecting software vulnerabilities using deep graph neural network. *ACM Trans. Soft. Eng. Meth.* **30**(3), 1–33 (2021)
- Cheng, X., Nie, X., Li, N., Wang, H., Zheng, Z., Sui, Y.: How about bug-triggering paths?-understanding and characterizing learning-based vulnerability detectors. *TDSC* **21**(2), 542–558 (2024)
- Croft, R., Babar, M.A.e.a.: Data quality for software vulnerability datasets. In: ICSE, pp. 121–133 (2023)
- CVE. (2025). <https://cve.mitre.org/>
- CWE. (2025). <https://cwe.mitre.org/>
- Das, S., Fabiha, S.T., Shafiq, S., Medvidović, N.: Are we learning the right features? a framework for evaluating dl-based software vulnerability detection solutions. In: Proceedings of the IEEE/ACM 47th International Conference on Software Engineering. ICSE '25, pp. 2893–290 (2025). <https://doi.org/10.1109/ICSE55347.2025.00194>
- Ding, Y., Fu, Y., Ibrahim, O., Sitawarin, C., Chen, X., Alomair, B., Wagner, D., Ray, B., Chen, Y.: Vulnerability Detection with Code Language Models: How Far Are We? . In: ICSE, CA, USA, pp. 469–481 (2025). <https://doi.org/10.1109/ICSE55347.2025.00038>
- Ding, Y., Suneja, S., Zheng, Y., Laredo, J., Morari, A., Kaiser, G., Ray, B.: Velvet: a novel ensemble learning approach to automatically locate vulnerable statements. In: SANER, pp. 959–970 (2022)
- Dolan-Gavitt, B., Hulin, P., Kirda, E., Leek, T.e.a.: Lava: Large-scale automated vulnerability addition. In: S&P, pp. 110–121 (2016)
- Elsayed, G.F., Goodfellow, I., Sohl-Dickstein, J.: Adversarial reprogramming of neural networks (2018)

- Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., et al.: Codebert: A pre-trained model for programming and natural languages. arXiv (2020)
- Fu, M., Tantithamthavorn, C.: Linevul: A transformer-based line-level vulnerability prediction. In: 19th Int. Conf. on Mining Software Repositories, pp. 608–620 (2022)
- Gao, S., Mao, W., Gao, C., Li, L., Hu, X., Xia, X., Lyu, M.R.: Learning in the wild: Towards leveraging unlabeled data for effectively tuning pre-trained code models. In: ICSE (2024). <https://doi.org/10.1145/3597503.3639216>
- Guo, D., Lu, S., Duan, N., Wang, Y., Zhou, M., Yin, J.: UniXcoder: Unified cross-modal pre-training for code representation. In: ACL, pp. 7212–7225 (2022)
- Guo, Y., Patsakis, C., Hu, Q., Tang, Q., Casino, F.: Outside the comfort zone: Analysing llm capabilities in software vulnerability detection. In: European Symposium on Research in Computer Security, pp. 271–289. Springer (2024)
- Guo, D., Ren, S., Lu, S., et al.: Graphcodebert: Pre-training code representations with data flow. arXiv (2020)
- Hanif, H., Maffei, S.: Vulberta: Simplified source code pre-training for vulnerability detection. In: IJCNN, pp. 1–8. IEEE (2022)
- Hin, D., Kan, A., Chen, H., Babar, M.A.: Linevd: statement-level vulnerability detection using graph neural networks. In: Proceedings of the 19th International Conference on Mining Software Repositories. MSR '22, pp. 596–607. Association for Computing Machinery, New York, NY, USA (2022). <https://doi.org/10.1145/3524842.3527949>
- Jia, X., Du, J., Fang, M., Liu, H., Li, Y., Jiang, F.: Vulnerability detection with graph enhancement and global dependency representation learning. Autom. Softw. Eng. **32**(1), 14 (2025). <https://doi.org/10.1007/s10515-024-00484-3>
- Lester, B., Al-Rfou, R., Constant, N.: The power of scale for parameter-efficient prompt tuning. In: EMNLP, pp. 3045–3059 (2021)
- Li, Q., Chen, L., Jing, S., Wu, D.: Knowledge distillation on cross-modal adversarial reprogramming for data-limited attribute inference. In: WWW, pp. 65–68 (2023)
- Li, X.L., Liang, P.: Prefix-tuning: Optimizing continuous prompts for generation. In: ACL-IJCNLP, pp. 4582–4597 (2021)
- Li, Z., Tang, J., Zou, D., Chen, Q., Xu, S., Zhang, C., Li, Y., Jin, H.: Towards making deep learning-based vulnerability detectors robust. arXiv (2021a)
- Li, Y., Wang, S., Nguyen, T.N.: Vulnerability detection with fine-grained interpretations. In: ESEC/FSE, pp. 292–303 (2021b)
- Li, Y., Zemel, R., Brockschmidt, M., Tarlow, D.: Gated graph sequence neural networks. In: Proceedings of ICLR'16 (2016)
- Li, Z., Zou, D., Xu, S., Jin, H., Zhu, Y., Chen, Z.: Sysevr: A framework for using deep learning to detect software vulnerabilities. TDSC, 2244–2258 (2021c)
- Li, Z., Zou, D., Xu, S., Ou, X., Jin, H., Wang, S., Deng, Z., Zhong, Y.: Vuldeepecker: A deep learning-based system for vulnerability detection. In: NDSS (2018)
- Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., et al.: Roberta: A robustly optimized bert pretraining approach. arXiv (2019)
- Lu, G., Ju, X., Chen, X., Pei, W., Cai, Z.: Grace: Empowering llm-based software vulnerability detection with graph structure and in-context learning. J. Syst. Softw. **212**(C) (2024). <https://doi.org/10.1016/j.jss.2024.112031>
- Mao, C., Zhong, Z., Yang, J., Vondrick, C., Ray, B.: Metric learning for adversarial robustness. Adv. Neural Inf. Process. Syst. **32** (2019)
- Neekhara, P., Hussain, S., Du, J., Dubnov, S., Koushanfar, F., McAuley, J.: Cross-modal adversarial reprogramming. In: WACV, pp. 2427–2435 (2022)
- Neekhara, P., Hussain, S., Dubnov, S., Koushanfar, F.: Adversarial reprogramming of text classification neural networks. In: EMNLP-IJCNLP, pp. 5216–5225 (2019)
- Niu, C., Li, C., Ng, V., Ge, J., Huang, L., Luo, B.: SPT-code: Sequence-to-sequence pre-training for learning source code representations. In: ICSE (2022)
- Nong, Y., Aldeen, M., Cheng, L., Hu, H., Chen, F., Cai, H.: Chain-of-Thought Prompting of Large Language Models for Discovering and Fixing Software Vulnerabilities (2024). <https://arxiv.org/abs/2402.17230>
- Nong, Y., Fang, R., Yi, G.e.a.: VGX: Large-scale sample generation for boosting learning-based software vulnerability analyses. arXiv (2023a)
- Nong, Y., Ou, Y., Pradel, M., Chen, F., Cai, H.: Generating realistic vulnerabilities via neural code editing: an empirical study. In: FSE, pp. 1097–1109 (2022)

- Nong, Y., Ou, Y., Pradel, M., Chen, F., Cai, H.: VULGEN: Realistic vulnerability generation via pattern mining and deep learning. In: ICSE (2023b)
- Rahman, M.M., Ceka, I., Mao, C., Chakraborty, S., Ray, B., Le, W.: Towards causal deep learning for vulnerability detection. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. ICSE '24. Association for Computing Machinery, New York, NY, USA (2024). <https://doi.org/10.1145/3597503.3639170>
- Rainio, O., Teuhio, J., Klen, R.: Evaluation metrics and statistical tests for machine learning. *Sci. Rep.* **14**(1), 6086 (2024). <https://doi.org/10.1038/s41598-024-56706-x>
- Ren, Z., Ju, X., Chen, X., Shen, H.: Prorlearn: boosting prompt tuning-based vulnerability detection by reinforcement learning. *Autom. Softw. Eng.* **31**(2), 38 (2024). <https://doi.org/10.1007/s10515-024-00438-9>
- Risse, N., Liu, J., Böhme, M.: Top score on the wrong exam: On benchmarking in machine learning for vulnerability detection. *Proc. ACM Softw. Eng.* **2**(ISSTA) (2025). <https://doi.org/10.1145/3728887>
- Russell, R., Kim, L., Hamilton, L., Lazovich, T., Harer, J., Ozdemir, O., Ellingwood, P., McConley, M.: Automated vulnerability detection in source code using deep representation learning. In: ICMLA, pp. 757–762 (2018)
- Schuster, M., Paliwal, K.K.: Bidirectional recurrent neural networks. *IEEE Trans. Signal Process.* **45**(11), 2673–2681 (1997)
- Shao, M., Ding, Y., Cao, J., Li, Y.: Graphfvd: Property graph-based fine-grained vulnerability detection. *Comput. Secur.* **151**, 104350 (2025)
- Steenhoek, B., Rahman, M.M., Jiles, R., Le, W.: An empirical study of deep learning models for vulnerability detection. In: ICSE, pp. 2237–2248 (2023)
- Steenhoek, B., Rahman, M.M., Roy, M.K., Alam, M.S., Barr, E.T.: A Comprehensive Study of the Capabilities of Large Language Models for Vulnerability Detection (2024). <https://arxiv.org/abs/2403.17218>
- Tang, Z., Hu, Q., Hu, Y., Kuang, W., Chen, J.: Sevuldet: A semantics-enhanced learnable vulnerability detector. In: DSN, pp. 150–162 (2022)
- Tian, Z., Li, H., Sun, H., Chen, Y., Chen, L.: Hardvd: High-capacity cross-modal adversarial reprogramming for data-efficient vulnerability detection. *Inf. Sci.* **686**(C) (2025). <https://doi.org/10.1016/j.ins.2024.121370>
- Tian, Z., Tian, B., Lv, J., Chen, Y., Chen, L.: Enhancing vulnerability detection via ast decomposition and neural sub-tree encoding. *ESWA* **238**, 121865 (2024)
- Tian, Z., Qiu, R., Teng, Y., Sun, J., Chen, Y., Chen, L.: Towards cost-efficient vulnerability detection with cross-modal adversarial reprogramming. *J. Syst. Softw.* **223**, 112365 (2025). <https://doi.org/10.1016/j.jss.2025.112365>
- Tree-sitter. (2025). <https://tree-sitter.github.io/tree-sitter/>
- Tsai, Y.-Y., Chen, P.-Y., Ho, T.-Y.: Transfer learning without knowing: Reprogramming black-box machine learning models with scarce data and limited resources. In: ICML, pp. 9614–9624 (2020)
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I.: Attention is all you need. *NeurIPS* **30** (2017)
- Wang, W., Nguyen, T.N., Wang, S., Li, Y., Zhang, J., Yadavally, A.: Deepvd: Toward class-separation features for neural network vulnerability detection. In: ICSE, pp. 2249–2261 (2023)
- Wen, X.-C., Chen, Y., Gao, C., Zhang, H., Zhang, J.M.: Vulnerability detection with graph simplification and enhanced graph representation learning. In: ICSE, pp. 2275–2286 (2023)
- Wen, X.-C., Gao, C., Gao, S., Xiao, Y., Lyu, M.R.: Scale: Constructing structured natural language comment trees for software vulnerability detection. In: 33rd Int. Sym. on Sof. Testing and Analysis. ISSTA 2024, pp. 235–247. New York, NY, USA (2024a). <https://doi.org/10.1145/3650212.3652124>
- Wen, X.-C., Gao, C., Luo, F., Wang, H., Li, G., Liao, Q.: Livable: Exploring long-tailed classification of software vulnerability types. *IEEE Trans. Software Eng.* **50**(6), 1325–1339 (2024b). <https://doi.org/10.1109/TSE.2024.3382361>
- Wu, T., Chen, L., Du, G., Meng, D., Shi, G.: Ultravcs: Ultra-fine-grained variable-based code slicing for automated vulnerability detection. *TIFS*, 1–1 (2024)
- Wu, Y., Zou, D., Dou, S., Yang, W., Xu, D., Jin, H.: Vulcnn: An image-inspired scalable vulnerability detection system. In: ICSE, pp. 2365–2376 (2022)
- Yang, X., Wang, S., Li, Y., Wang, S.: Does data sampling improve deep learning-based vulnerability detection? yeas! and nays! In: ICSE, pp. 2287–2298 (2023)
- Yefet, N., Alon, U., Yahav, E.: Adversarial examples for models of code. *OOPSLA* **4**, 1–30 (2020)
- Yin, X., Ni, C., Wang, S.: Multitask-based evaluation of open-source llm on software vulnerability. *IEEE Trans. Softw. Eng.* **50**(11), 3071–3087 (2024). <https://doi.org/10.1109/TSE.2024.3470333>

- Yu, X., Li, Z., Huang, X., Zhao, S.: Advulcode: Generating adversarial vulnerable code against deep learning-based vulnerability detectors. *Electronics* **12**(4) (2023). <https://doi.org/10.3390/electronics12040936>
- Zhang, H., Li, Z., Li, G., Ma, L., Liu, Y., Jin, Z.: Generating adversarial examples for holding robustness of source code processing models. In: AAAI, vol. 34, pp. 1169–1176 (2020)
- Zhang, J., Liu, Z., Hu, X., Xia, X., Li, S.: Vulnerability detection by learning from syntax-based execution paths of code. *IEEE Trans. Softw. Eng.* **49**(8), 4196–4212 (2023). <https://doi.org/10.1109/TSE.2023.3286586>
- Zheng, T., Liu, H., Xu, H., Chen, X., Yi, P., Wu, Y.: Few-vuld: A few-shot learning framework for software vulnerability detection. *Comput. Secur.* **144**, 103992 (2024). <https://doi.org/10.1016/j.cose.2024.103992>
- Zhou, Y., Liu, S., Siow, J., Du, X., Liu, Y.: Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. *NeurIPS* **32** (2019)
- Zhou, X., Cao, S., Sun, X., Lo, D.: Large language model for vulnerability detection and repair: Literature review and the road ahead. *ACM Trans. Softw. Eng. Methodol.* (2024). <https://doi.org/10.1145/3708522>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

Zhenzhou Tian^{1,2,3} · Chuang Zhang¹ · Yunpeng Hui¹ · Jiaze Sun^{1,2,3} · Yanping Chen^{1,2,3} · Lingwei Chen⁴

✉ Zhenzhou Tian
tianzhenzhou@xupt.edu.cn

Chuang Zhang
zclgmn@163.com

Yunpeng Hui
yunpengxi@stu.xupt.edu.cn

Jiaze Sun
sunjiaze@xupt.edu.cn

Yanping Chen
chenyp@xupt.edu.cn

Lingwei Chen
lwcics@rit.edu

¹ School of Computer Science and Technology, Xi'an University of Posts and Telecommunications, Western Chang'an Street, Xi'an 710121, Shaanxi, China

² Shaanxi Key Laboratory of Network Data Analysis and Intelligent Processing, Western Chang'an Street, Xi'an 710121, Shaanxi, China

³ Xi'an Key Laboratory of Big Data and Intelligent Computing, Western Chang'an Street, Xi'an 710121, Shaanxi, China

⁴ Department of Cybersecurity, Rochester Institute of Technology, 1 Lomb Memorial Dr, Rochester, NY 14623, USA